Physlib Monthly Report

1–31 December 2024

Contributors: Joseph Tooby-Smith

Generated 14 August 2026 from commit `abfda8f`.

Abstract

Physlib is a community library of formalised physics for the Lean 4 proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month's additions to it.

It covers 1–31 December 2024 on branch `master` of `leanprover-community/physlib`, between commits `a57d38c` and `abfda8f`. Over that period 1 contributor landed 115 commits, changing 6,747 lines across 150 files and adding 274 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at GitHub compare.

Contents

1 Introduction

During Dec 2024, 1 contributor submitted 115 commits that changed 6,747 lines (+5,053 / -1,694) across 150 files spanning 20 top-level areas of the repository. Of those, 24 files were newly added and 4 were removed outright. This report catalogues 274 newly added Lean declarations: 176 lemmas, 61 defs, 23 instances, 4 informal definitions, 4 structures, 3 theorems, 2 inductives, and 1 class.

See the full compare diff and the list of commits on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Joseph Tooby-Smith	21,343	115

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

No reviewers identified for this month.

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
`HepLean/PerturbationTheory`	18	+14	–	+2,952	-11
`HepLean/FeynmanDiagrams`	4	–	-4	+0	-975
`HepLean/Mathematics`	6	+2	–	+888	-27
`HepLean/Meta`	8	+5	–	+624	-45
`HepLean/Tensors`	21	–	–	+258	-228
`HepLean/Lorentz`	32	–	–	+51	-155
`lake-manifest.json`	1	–	–	+72	-62
`scripts/MetaPrograms`	4	+2	–	+72	-26
`HepLean/AnomalyCancellation`	33	–	–	+1	-91
`HepLean/StandardModel`	6	–	–	+36	-38
`HepLean.lean`	1	–	–	+26	-9
`scripts`	2	+1	–	+30	-1
`lakefile.toml`	1	–	–	+23	-3
`HepLean/FlavorPhysics`	4	–	–	+0	-12
`HepLean/BeyondTheStandardModel`	4	–	–	+3	-7
`README.md`	1	–	–	+7	-2
`docs`	1	–	–	+6	-0
`.github/workflows`	1	–	–	+3	-0
`lean-toolchain`	1	–	–	+1	-1
`HepLean/SpaceTime`	1	–	–	+0	-1

1.4 New declarations by kind

Kind	Count
lemma	176
def	61
instance	23
informal_definition	4
structure	4
theorem	3
inductive	2
class	1

2 Details by file

2.1 HepLean.Mathematics.Fin

HepLean/Mathematics/Fin.lean on GitHub.

lemma finExtractOne_apply_neq [source]

```
lemma finExtractOne_apply_neq {n : ℕ} (i j : Fin n.succ.succ) (hij : i ≠ j) :
    finExtractOne i j = Sum.inr (predAboveI i j)
```

lemma finExtractOnePerm_equiv [source]

```
lemma finExtractOnePerm_equiv {n m : ℕ} (e : Fin n.succ.succ ≃ Fin m.succ.succ)
    (i : Fin n.succ.succ) :
    e ∘ i.succAbove = (e i).succAbove ∘ finExtractOnePerm i e
```

def equivCons [source]

Given an equivalence between `Fin n` and `Fin m`, the induced equivalence between `Fin n.succ` and `Fin m.succ` derived by `Fin.cons`.

```
def equivCons {n m : ℕ} (e : Fin n ≃ Fin m) : Fin n.succ ≃ Fin m.succ where
```

lemma equivCons_zero [source]

```
@[simp]
lemma equivCons_zero {n m : ℕ} (e : Fin n ≃ Fin m) :
    equivCons e 0 = 0
```

lemma equivCons_trans [source]

```
@[simp]
lemma equivCons_trans {n m k : ℕ} (e : Fin n ≃ Fin m) (f : Fin m ≃ Fin k) :
    Fin.equivCons (e.trans f) = (Fin.equivCons e).trans (Fin.equivCons f)
```

lemma equivCons_castOrderIso [source]

```
@[simp]
lemma equivCons_castOrderIso {n m : ℕ} (h : n = m) :
    (Fin.equivCons (Fin.castOrderIso h).toEquiv) = (Fin.castOrderIso (by simp [h])).toEquiv
```

lemma equivCons_symm_succ [source]

```
@[simp]
lemma equivCons_symm_succ {n m : ℕ} (e : Fin n ≃ Fin m) (i : ℕ) (hi : i + 1 < m.succ) :
    (Fin.equivCons e).symm ⟨i + 1, hi⟩ = (e.symm ⟨i, Nat.succ_lt_succ_iff.mp hi⟩).succ
```

lemma equivCons_succ [source]

```
@[simp]
lemma equivCons_succ {n m : ℕ} (e : Fin n ≃ Fin m) (i : ℕ) (hi : i + 1 < n.succ) :
    (Fin.equivCons e) ⟨i + 1, hi⟩ = (e ⟨i, Nat.succ_lt_succ_iff.mp hi⟩).succ
```

2.2 HepLean.Mathematics.List

HepLean/Mathematics/List.lean on GitHub.

lemma takeWile_eraseIdx [source]

```
lemma takeWile_eraseIdx {I : Type} (P : I → Prop) [DecidablePred P] :
    (l : List I) → (i : ℕ) → (hi : ∀ (i j : Fin l.length), i < j → P (l.get j) → P (l.get i)) →
    List.takeWhile P (List.eraseIdx l i) = (List.takeWhile P l).eraseIdx i
  | [], _, h => by
    rfl
  | a :: [], 0, h => by
    simp only [List.takeWhile, List.eraseIdx_zero, List.nil_eq]
    split
    · rfl
    · rfl
  | a :: [], Nat.succ n, h => by
    simp only [Nat.succ_eq_add_one, List.eraseIdx_cons_succ, List.eraseIdx_nil]
    rw [List.eraseIdx_of_length_le]
    have h1 : (List.takeWhile P [a]).length ≤ [a].length
```

lemma dropWile_eraseIdx [source]

```
lemma dropWile_eraseIdx {I : Type} (P : I → Prop) [DecidablePred P] :
    (l : List I) → (i : ℕ) → (hi : ∀ (i j : Fin l.length), i < j → P (l.get j) → P (l.get i)) →
    List.dropWhile P (List.eraseIdx l i) =
      if (List.takeWhile P l).length ≤ i then
        (List.dropWhile P l).eraseIdx (i - (List.takeWhile P l).length)
      else (List.dropWhile P l)
  | [], _, h => by
    simp
  | a :: [], 0, h => by
    simp only [List.dropWhile, nonpos_iff_eq_zero, List.length_eq_zero, List.
    takeWhile_eq_nil_iff,
      List.length_singleton, zero_lt_one, Fin.zero_eta, Fin.isValue, List.get_eq_getElem,
      Fin.val_eq_zero, List.getElem_cons_zero, decide_eq_true_eq, forall_const, zero_le,
      Nat.sub_eq_zero_of_le, List.eraseIdx_zero, ite_not, List.nil_eq]
    simp_all only [List.length_singleton, List.get_eq_getElem, Fin.val_eq_zero,
      List.getElem_cons_zero, implies_true, decide_True, decide_False, List.tail_cons, ite_self
    ]
  | a :: [], Nat.succ n, h => by
    simp only [List.dropWhile, List.eraseIdx_nil, List.takeWhile, Nat.succ_eq_add_one]
    rw [List.eraseIdx_of_length_le]
    simp_all only [List.length_singleton, List.get_eq_getElem, Fin.val_eq_zero,
```

```
      List.getElem_cons_zero, implies_true, ite_self]
    simp_all only [List.length_singleton, List.get_eq_getElem, Fin.val_eq_zero,
      List.getElem_cons_zero, implies_true]
    split
    next x heq =>
      simp_all only [decide_eq_true_eq, List.length_nil, List.length_singleton,
        add_tsub_cancel_right, zero_le]
    next x heq =>
      simp_all only [decide_eq_false_iff_not, List.length_singleton, List.length_nil, tsub_zero
    ,
        le_add_iff_nonneg_left, zero_le]
  | a :: b :: l, 0, h => by
    simp only [List.dropWhile, List.takeWhile, nonpos_iff_eq_zero, List.length_eq_zero, zero_le
    ,
      Nat.sub_eq_zero_of_le, List.eraseIdx_zero]
    by_cases hPb : P b
    · have hPa : P a
```

def orderedInsertPos [source]

The position `r0` ends up in `r` on adding it via `List.orderedInsert _ r0 r`.

```
def orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) :
    Fin (List.orderedInsert le1 r0 r).length
```

lemma orderedInsertPos_lt_length [source]

```
lemma orderedInsertPos_lt_length {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I
    )
    (r0 : I) : orderedInsertPos le1 r r0 < (r0 :: r).length
```

lemma orderedInsert_get_orderedInsertPos [source]

```
@[simp]
lemma orderedInsert_get_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r : List I) (r0 : I) :
    (List.orderedInsert le1 r0 r)[(orderedInsertPos le1 r r0).val] = r0
```

lemma orderedInsert_eraseIdx_orderedInsertPos [source]

```
@[simp]
lemma orderedInsert_eraseIdx_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel le1
    ]
    (r : List I) (r0 : I) :
    (List.orderedInsert le1 r0 r).eraseIdx ↑(orderedInsertPos le1 r r0) = r
```

lemma orderedInsertPos_cons [source]

```
lemma orderedInsertPos_cons {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r : List I) (r0 r1 : I) :
    (orderedInsertPos le1 (r1 ::r) r0).val =
    if le1 r0 r1 then ⟨0, by simp⟩ else (Fin.succ (orderedInsertPos le1 r r0))
```

lemma orderedInsertPos_sigma [source]

```
lemma orderedInsertPos_sigma {I : Type} {f : I → Type}
    (le1 : I → I → Prop) [DecidableRel le1] (l : List (Σ i, f i))
    (k : I) (a : f k) :
    (orderedInsertPos (fun (i j : Σ i, f i) => le1 i.1 j.1) l ⟨k, a⟩).1 =
    (orderedInsertPos le1 (List.map (fun (i : Σ i, f i) => i.1) l) k).1
```

lemma orderedInsert_get_lt [source]

```
lemma orderedInsert_get_lt {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r : List I) (r0 : I) (i : ℕ)
    (hi : i < orderedInsertPos le1 r r0) :
    (List.orderedInsert le1 r0 r)[i] = r.get ⟨i, by
      simp only [orderedInsertPos] at hi
      have h1 : (List.takeWhile (fun b => decide ¬le1 r0 b) r).length ≤ r.length :=
        List.Sublist.length_le (List.takeWhile_sublist fun b => decide ¬le1 r0 b)
      omega⟩
```

lemma orderedInsertPos_take_orderedInsert [source]

```
lemma orderedInsertPos_take_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r : List I) (r0 : I) :
    (List.take (orderedInsertPos le1 r r0) (List.orderedInsert le1 r0 r)) =
    List.takeWhile (fun b => decide ¬le1 r0 b) r
```

lemma orderedInsertPos_take_eq_orderedInsert [source]

```
lemma orderedInsertPos_take_eq_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r : List I) (r0 : I) :
    List.take (orderedInsertPos le1 r r0) r =
    List.take (orderedInsertPos le1 r r0) (List.orderedInsert le1 r0 r)
```

lemma orderedInsertPos_drop_eq_orderedInsert [source]

```
lemma orderedInsertPos_drop_eq_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r : List I) (r0 : I) :
    List.drop (orderedInsertPos le1 r r0) r =
    List.drop (orderedInsertPos le1 r r0).succ (List.orderedInsert le1 r0 r)
```

lemma orderedInsertPos_take [source]

```
lemma orderedInsertPos_take {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r : List I) (r0 : I) :
    List.take (orderedInsertPos le1 r r0) r = List.takeWhile (fun b => decide ¬le1 r0 b) r
```

lemma orderedInsertPos_drop [source]

```
lemma orderedInsertPos_drop {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r : List I) (r0 : I) :
    List.drop (orderedInsertPos le1 r r0) r = List.dropWhile (fun b => decide ¬le1 r0 b) r
```

lemma orderedInsertPos_succ_take_orderedInsert [source]

```
lemma orderedInsertPos_succ_take_orderedInsert {I : Type} (le1 : I → I → Prop) [DecidableRel
    le1]
    (r : List I) (r0 : I) :
    (List.take (orderedInsertPos le1 r r0).succ (List.orderedInsert le1 r0 r)) =
    List.takeWhile (fun b => decide ¬le1 r0 b) r ++ [r0]
```

lemma lt_orderedInsertPos_rel [source]

```
lemma lt_orderedInsertPos_rel {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    (r0 : I) (r : List I) (n : Fin r.length)
    (hn : n.val < (orderedInsertPos le1 r r0).val) : ¬ le1 r0 (r.get n)
```

lemma gt_orderedInsertPos_rel [source]

```
lemma gt_orderedInsertPos_rel {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    [IsTotal I le1] [IsTrans I le1] (r0 : I) (r : List I) (hs : List.Sorted le1 r)
    (n : Fin r.length)
    (hn : ¬ n.val < (orderedInsertPos le1 r r0).val) : le1 r0 (r.get n)
```

lemma orderedInsert_eraseIdx_lt_orderedInsertPos [source]

```
lemma orderedInsert_eraseIdx_lt_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel
    le1]
    (r : List I) (r0 : I) (i : ℕ)
    (hi : i < orderedInsertPos le1 r r0)
    (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
    (List.orderedInsert le1 r0 r).eraseIdx i = List.orderedInsert le1 r0 (r.eraseIdx i)
```

lemma orderedInsert_eraseIdx_orderedInsertPos_le [source]

```
lemma orderedInsert_eraseIdx_orderedInsertPos_le {I : Type} (le1 : I → I → Prop) [DecidableRel
    le1]
    (r : List I) (r0 : I) (i : ℕ)
    (hi : orderedInsertPos le1 r r0 ≤ i)
    (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
    (List.orderedInsert le1 r0 r).eraseIdx (Nat.succ i) =
    List.orderedInsert le1 r0 (r.eraseIdx i)
```

def orderedInsertEquiv [source]

The equivalence between `Fin (r0 :: r).length` and `Fin (List.orderedInsert le1 r0 r).length` according to where the elements map. I.e. `0` is taken to `orderedInsertPos le1 r r0`.

```
def orderedInsertEquiv {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I)
    :
    Fin (r0 :: r).length ≃ Fin (List.orderedInsert le1 r0 r).length
```

lemma orderedInsertEquiv_zero [source]

```
lemma orderedInsertEquiv_zero {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
    (r0 : I) : orderedInsertEquiv le1 r r0 (0, by simp) = orderedInsertPos le1 r r0
```

lemma orderedInsertEquiv_succ [source]

```
lemma orderedInsertEquiv_succ {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
    (r0 : I) (n : ℕ) (hn : Nat.succ n < (r0 :: r).length) :
    orderedInsertEquiv le1 r r0 ⟨Nat.succ n, hn⟩ =
    Fin.cast (List.orderedInsert_length le1 r r0).symm
    ((Fin.succAbove ⟨(orderedInsertPos le1 r r0), orderedInsertPos_lt_length le1 r r0⟩)
    ⟨n, Nat.succ_lt_succ_iff.mp hn⟩)
```

lemma orderedInsertEquiv_fin_succ [source]

```
lemma orderedInsertEquiv_fin_succ {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List
    I)
    (r0 : I) (n : Fin r.length) :
    orderedInsertEquiv le1 r r0 n.succ = Fin.cast (List.orderedInsert_length le1 r r0).symm
    ((Fin.succAbove ⟨(orderedInsertPos le1 r r0), orderedInsertPos_lt_length le1 r r0⟩)
      ⟨n, n.isLt⟩)
```

lemma orderedInsertEquiv_congr [source]

```
lemma orderedInsertEquiv_congr {α : Type} {r : α → α → Prop} [DecidableRel r] (a : α)
    (l l' : List α) (h : l = l') :
    orderedInsertEquiv r l a = (Fin.castOrderIso (by simp [h])).toEquiv.trans
    ((orderedInsertEquiv r l' a).trans (Fin.castOrderIso (by simp [h])).toEquiv)
```

lemma get_eq_orderedInsertEquiv [source]

```
lemma get_eq_orderedInsertEquiv {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
    (r0 : I) :
    (r0 :: r).get = (List.orderedInsert le1 r0 r).get ∘ (orderedInsertEquiv le1 r r0)
```

lemma orderedInsertEquiv_get [source]

```
lemma orderedInsertEquiv_get {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I)
    (r0 : I) :
    (r0 :: r).get ∘ (orderedInsertEquiv le1 r r0).symm = (List.orderedInsert le1 r0 r).get
```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_zero [source]

```
lemma orderedInsert_eraseIdx_orderedInsertEquiv_zero
    {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) :
    (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 ⟨0, by simp⟩) = r
```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_succ [source]

```
lemma orderedInsert_eraseIdx_orderedInsertEquiv_succ
    {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) (n : ℕ)
    (hn : Nat.succ n < (r0 :: r).length)
    (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
    (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 ⟨Nat.succ n, hn⟩) =
    (List.orderedInsert le1 r0 (r.eraseIdx n))
```

lemma orderedInsert_eraseIdx_orderedInsertEquiv_fin_succ [source]

```
lemma orderedInsert_eraseIdx_orderedInsertEquiv_fin_succ
    {I : Type} (le1 : I → I → Prop) [DecidableRel le1] (r : List I) (r0 : I) (n : Fin r.length)
    (hr : ∀ (i j : Fin r.length), i < j → ¬le1 r0 (r.get j) → ¬le1 r0 (r.get i)) :
    (List.orderedInsert le1 r0 r).eraseIdx (orderedInsertEquiv le1 r r0 n.succ) =
    (List.orderedInsert le1 r0 (r.eraseIdx n))
```

lemma orderedInsertEquiv_sigma [source]

```
lemma orderedInsertEquiv_sigma {I : Type} {f : I → Type}
    (le1 : I → I → Prop) [DecidableRel le1] (l : List (Σ i, f i))
    (i : I) (a : f i) :
    (orderedInsertEquiv (fun i j => le1 i.fst j.fst) l ⟨i, a⟩) =
    (Fin.castOrderIso (by simp)).toEquiv.trans
    ((orderedInsertEquiv le1 (List.map (fun i => i.1) l) i).trans
    (Fin.castOrderIso (by simp [List.orderedInsert_length])).toEquiv)
```

theorem length_insertIdx' [source]

This result is taken from: https://github.com/leanprover/lean4/blob/master/src/Init/Data/List/Nat/Inse with simple modification here to make it run. The file it was taken from is licensed under the Apache License, Version 2.0. and written by Parikshit Khanna, Jeremy Avigad, Leonardo de Moura, Floris van Doorn, Mario Carneiro.

Once HepLean is updated to a more recent version of Lean this result will be removed.

```
theorem length_insertIdx' : ∀ n as, (List.insertIdx n a as).length =
    if n ≤ as.length then as.length + 1 else as.length
  | 0, _ => by simp
  | n + 1, [] => by rfl
  | n + 1, a :: as => by
    simp only [List.insertIdx_succ_cons, List.length_cons, length_insertIdx',
      Nat.add_le_add_iff_right]
    split <;> rfl
```

theorem getElem_insertIdx_of_ge [source]

This result is taken from: https://github.com/leanprover/lean4/blob/master/src/Init/Data/List/Nat/Inse with simple modification here to make it run. The file it was taken from is licensed under the Apache License, Version 2.0. and written by Parikshit Khanna, Jeremy Avigad, Leonardo de Moura, Floris van Doorn, Mario Carneiro.

Once HepLean is updated to that version of Lean this result will be removed.

```
theorem _root_.List.getElem_insertIdx_of_ge {l : List α} {x : α} {n k : Nat} (hn : n + 1 ≤ k)
    (hk : k < (List.insertIdx n x l).length) :
    (List.insertIdx n x l)[k] =
    l[k - 1]'(by simp only [length_insertIdx'] at hk; split at hk <;> omega)
```

lemma orderedInsert_eq_insertIdx_orderedInsertPos [source]

```
lemma orderedInsert_eq_insertIdx_orderedInsertPos {I : Type} (le1 : I → I → Prop) [DecidableRel
     le1]
    (r : List I) (r0 : I) :
    List.orderedInsert le1 r0 r = List.insertIdx (orderedInsertPos le1 r r0).1 r0 r
```

def insertionSortEquiv [source]

The equivalence between `Fin l.length ≃ Fin (List.insertionSort r l).length` induced by the sorting algorithm.

```
def insertionSortEquiv {α : Type} (r : α → α → Prop) [DecidableRel r] : (l : List α) →
    Fin l.length ≃ Fin (List.insertionSort r l).length
  | [] => Equiv.refl _
  | a :: l =>
    (Fin.equivCons (insertionSortEquiv r l)).trans (orderedInsertEquiv r (List.insertionSort r
    l) a)
```

lemma insertionSortEquiv_get [source]

```
lemma insertionSortEquiv_get {α : Type} {r : α → α → Prop} [DecidableRel r] : (l : List α) →
    l.get ∘ (insertionSortEquiv r l).symm = (List.insertionSort r l).get
  | [] => by rfl
  | a :: l => by
    rw [insertionSortEquiv]
    change ((a :: l).get ∘ ((Fin.equivCons (insertionSortEquiv r l))).symm) ∘
      (orderedInsertEquiv r (List.insertionSort r l) a).symm = _
    have hl : (a :: l).get ∘ ((Fin.equivCons (insertionSortEquiv r l))).symm =
        (a :: List.insertionSort r l).get
```

lemma insertionSortEquiv_congr [source]

```
lemma insertionSortEquiv_congr {α : Type} {r : α → α → Prop} [DecidableRel r] (l l' : List α)
    (h : l = l') : insertionSortEquiv r l = (Fin.castOrderIso (by simp [h])).toEquiv.trans
      ((insertionSortEquiv r l').trans (Fin.castOrderIso (by simp [h])).toEquiv)
```

lemma insertionSort_get_comp_insertionSortEquiv [source]

```
lemma insertionSort_get_comp_insertionSortEquiv {α : Type} {r : α → α → Prop} [DecidableRel r]
    (l : List α) : (List.insertionSort r l).get ∘ (insertionSortEquiv r l) = l.get
```

lemma insertionSort_eq_ofFn [source]

```
lemma insertionSort_eq_ofFn {α : Type} {r : α → α → Prop} [DecidableRel r] (l : List α) :
    List.insertionSort r l = List.ofFn (l.get ∘ (insertionSortEquiv r l).symm)
```

def optionErase [source]

Optional erase of an element in a list. For `none` returns the list, for `some i` returns the list with the `i`'th element erased.

```
def optionErase {I : Type} (l : List I) (i : Option (Fin l.length)) : List I
```

def optionEraseZ [source]

Optional erase of an element in a list, with addition for `none`. For `none` adds `a` to the front of the list, for `some i` removes the `i`th element of the list (does not add `a`). E.g. `optionEraseZ [0, 1, 2] 4 none = [4, 0, 1, 2]` and `optionEraseZ [0, 1, 2] 4 (some 1) = [0, 2]`.

```
def optionEraseZ {I : Type} (l : List I) (a : I) (i : Option (Fin l.length)) : List I
```

lemma eraseIdx_length [source]

```
lemma eraseIdx_length {I : Type} (l : List I) (i : Fin l.length) :
    (List.eraseIdx l i).length + 1 = l.length
```

lemma eraseIdx_cons_length [source]

```
lemma eraseIdx_cons_length {I : Type} (a : I) (l : List I) (i : Fin (a :: l).length) :
    (List.eraseIdx (a :: l) i).length= l.length
```

lemma eraseIdx_get [source]

```
lemma eraseIdx_get {I : Type} (l : List I) (i : Fin l.length) :
    (List.eraseIdx l i).get = l.get ∘ (Fin.cast (eraseIdx_length l i)) ∘
    (Fin.cast (eraseIdx_length l i).symm i).succAbove
```

lemma eraseIdx_insertionSort [source]

```
lemma eraseIdx_insertionSort {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    [IsTotal I le1] [IsTrans I le1] :
    (n : ℕ) → (r : List I) → (hn : n < r.length) →
    (List.insertionSort le1 r).eraseIdx ↑((HepLean.List.insertionSortEquiv le1 r) ⟨n, hn⟩)
    = List.insertionSort le1 (r.eraseIdx n)
  | 0, [], _ => by rfl
  | 0, (r0 :: r), hn => by
    simp only [List.insertionSort, List.insertionSort.eq_2, List.length_cons,
    insertionSortEquiv,
      Nat.succ_eq_add_one, Fin.zero_eta, Equiv.trans_apply, equivCons_zero, List.eraseIdx_zero,
      List.tail_cons]
    erw [orderedInsertEquiv_zero]
    simp
  | Nat.succ n, [], hn => by rfl
  | Nat.succ n, (r0 :: r), hn => by
    simp only [List.insertionSort, List.length_cons, insertionSortEquiv, Nat.succ_eq_add_one,
      Equiv.trans_apply, equivCons_succ]
    have hOr
```

lemma eraseIdx_insertionSort_fin [source]

```
lemma eraseIdx_insertionSort_fin {I : Type} (le1 : I → I → Prop) [DecidableRel le1]
    [IsTotal I le1] [IsTrans I le1] (r : List I) (n : Fin r.length) :
    (List.insertionSort le1 r).eraseIdx ↑((HepLean.List.insertionSortEquiv le1 r) n)
    = List.insertionSort le1 (r.eraseIdx n)
```

2.3 HepLean.Mathematics.SuperAlgebra.Basic

HepLean/Mathematics/SuperAlgebra/Basic.lean on GitHub.

informal_definition SuperAlgebra [source]

```
informal_definition SuperAlgebra where
  math :≈ "A super algebra is a graded algebra A with a ℤ₂ grading."
  physics :≈ "A super algebra is used to model the commutator of fermionic operators among
    themselves, aswell as among bosonic operators."
  ref :≈ "https://en.wikipedia.org/wiki/Superalgebra"
```

informal_definition superCommuator [source]

```
informal_definition superCommuator where
  math :≈ "The commutator which for `a ∈ Aᵢ` and `b ∈ Aⱼ` is defined as `ab - (-1)^(i * j) ba`.
    "
  deps :≈ [``SuperAlgebra]
```

2.4 HepLean.Meta.Basic

HepLean/Meta/Basic.lean on GitHub.

def toGitHubLink [source]

Turns a name, which represents a module, into a link to github.

```
def Name.toGitHubLink (c : Name) (l : Nat := 0) : MetaM String
```

def fileName [source]

Given a name, returns the file name corresponding to that declaration.

```
def Name.fileName (c : Name) : MetaM Name
```

def getDeclString [source]

Given a name, returns the source code defining that name.

```
def Name.getDeclString (name : Name) : MetaM String
```

2.5 HepLean.Meta.Informal.Post

HepLean/Meta/Informal/Post.lean on GitHub.

def constantInfoToInformalLemma [source]

Takes a `ConstantInfo` corresponding to a `InformalLemma` and returns the corresponding `InformalLemma`.

```
unsafe def constantInfoToInformalLemma (c : ConstantInfo) : MetaM InformalLemma
```

def constantInfoToInformalDefinition [source]

Takes a `ConstantInfo` corresponding to a `InformalDefinition` and returns the corresponding `InformalDefinition`.

```
unsafe def constantInfoToInformalDefinition (c : ConstantInfo) : MetaM InformalDefinition
```

2.6 HepLean.Meta.Notes.Basic

HepLean/Meta/Notes/Basic.lean on GitHub.

structure NoteInfo [source]

The information from a `note ...` command. To be used in a note file

```
structure NoteInfo where
  content : String
  fileName : Name
  line : Nat
```

def elabNote [source]

Elaborator for the `note ...` command

```
@[command_elab note_comment]
def elabNote : Lean.Elab.Command.CommandElab
```

2.7 HepLean.Meta.Notes.HTMLNote

HepLean/Meta/Notes/HTMLNote.lean on GitHub.

structure HTMLNote [source]

A `HTMLNote` is a structure containing the html information from invidual contributions (commands, informal commands, note ..) etc. to a note file.

```
structure HTMLNote where
  fileName : Name
  content : String
  line : Nat
```

def ofNodeInfo [source]

Converts a note infor into a HTMLNote.

```
def HTMLNote.ofNodeInfo (ni : NoteInfo) : MetaM HTMLNote
```

def ofFormal [source]

An formal definition or lemma to html for a note.

```
def HTMLNote.ofFormal (name : Name) : MetaM HTMLNote
```

def ofInformal [source]

An informal definition or lemma to html for a note.

```
unsafe def HTMLNote.ofInformal (name : Name) : MetaM HTMLNote
```

2.8 HepLean.Meta.Notes.NoteFile

HepLean/Meta/Notes/NoteFile.lean on GitHub.

structure NoteFile [source]

A note consists of a title and a list of Lean files which make up the note.

```
structure NoteFile where
  title : String
  abstract : String
  authors : List String
  files : List Name
```

def imports [source]

All imports associated to a note.

```
def imports : Array Import
```

2.9 HepLean.Meta.Notes.ToHTML

HepLean/Meta/Notes/ToHTML.lean on GitHub.

def sortLE [source]

Sorts `NoteInfo`'s by file name then by line number.

```
def sortLE (ni1 ni2 : HTMLNote) : Bool
```

def getNodeInfo [source]

Returns a sorted list of NodeInfos for a file system.

```
unsafe def getNodeInfo : MetaM (List HTMLNote)
```

def codeBlockHTML [source]

The HTML code needed to have syntax highlighting.

```
def codeBlockHTML : String
```

def informalDefStyle [source]

The html styles for informal definitions.

```
def informalDefStyle : String
```

def codeButton [source]

The html styles for code button.

```
def codeButton : String
```

def mathJaxScript [source]

HTML allowing the use of mathjax.

```
def mathJaxScript : String
```

def headerHTML [source]

The header to the html code.

```
def headerHTML : String
```

def titleHTML [source]

The html code corresponding to the title, abstract and authors.

```
def titleHTML : String
```

def leanNote [source]

The html code corresponding to a note about Lean and its use.

```
def leanNote : String
```

def footerHTML [source]

The footor of the html file.

```
def footerHTML : String
```

def toHTMLString [source]

The html file associated to a NoteFile string.

```
unsafe def toHTMLString : MetaM String
```

2.10 HepLean.PerturbationTheory.Contractions.Basic

HepLean/PerturbationTheory/Contractions/Basic.lean on GitHub.

inductive ContractionsAux [source]

Given a list of fields `l`, the type of pairwise-contractions associated with `l` which have the list `aux` uncontracted.

```
inductive ContractionsAux : (l : List 𝓕) → (aux : List 𝓕) → Type
  | nil : ContractionsAux [] []
  | cons {l : List 𝓕} {aux : List 𝓕} {a : 𝓕} (i : Option (Fin aux.length)) :
    ContractionsAux l aux → ContractionsAux (a :: l) (optionEraseZ aux a i)
```

def Contractions [source]

Given a list of fields `l`, the type of pairwise-contractions associated with `l`.

```
def Contractions (l : List 𝓕) : Type
```

def normalize [source]

The list of uncontracted fields.

```
def normalize : List 𝓕
```

lemma contractions_nil [source]

```
lemma contractions_nil (a : Contractions ([] : List 𝓕)) : a = ⟨[], ContractionsAux.nil⟩
```

lemma contractions_single [source]

Establishes uniqueness of contractions for a single field, showing that any contraction of a single field must be equivalent to the trivial contraction with no pairing.

```
lemma contractions_single {i : 𝓕} (a : Contractions [i]) : a =
    ⟨[i], ContractionsAux.cons none ContractionsAux.nil⟩
```

def nilEquiv [source]

This function provides an equivalence between the type of contractions for an empty list of fields and the unit type, indicating that there is only one possible contraction for an empty list.

```
def nilEquiv : Contractions ([] : List 𝓕) ≃ Unit where
```

def consEquiv [source]

The equivalence between contractions of `a :: l` and contractions of `Contractions l` paired with an optional element of `Fin (c.normalize).length` specifying what `a` contracts with if any.

```
def consEquiv {a : 𝓕} {l : List 𝓕} :
    Contractions (a :: l) ≃ (c : Contractions l) × Option (Fin (c.normalize).length) where
```

instance decidable [source]

The type of contractions is decidable.

```
instance decidable : (l : List 𝓕) → DecidableEq (Contractions l)
  | [] => fun a b =>
    match a, b with
    | ⟨_, a⟩, ⟨_, b⟩ =>
    match a, b with
    | ContractionsAux.nil, ContractionsAux.nil => isTrue rfl
  | _ :: l =>
    haveI : DecidableEq (Contractions l)
```

instance fintype [source]

The type of contractions is finite.

```
instance fintype : (l : List 𝓕) → Fintype (Contractions l)
  | [] => {
    elems := {⟨[], ContractionsAux.nil⟩}
    complete := by
      intro a
      rw [Finset.mem_singleton]
      exact contractions_nil a}
  | a :: l =>
    haveI : Fintype (Contractions l)
```

def IsFull [source]

A contraction is a full contraction if there normalizing list of fields is empty.

```
def IsFull : Prop
```

instance isFull_decidable [source]

Provides a decidable instance for determining if a contraction is full (i.e., all fields are paired).

```
instance isFull_decidable : Decidable c.IsFull
```

structure Splitting [source]

A structure specifying when a type `I` and a map `f :I → Type` corresponds to the splitting of a fields `φ` into a creation `n` and annihlation part `p`.

```
structure Splitting (f : 𝓕 → Type) [∀ i, Fintype (f i)]
    (le1 : (Σ i, f i) → (Σ i, f i) → Prop) [DecidableRel le1] where
  𝓑n : 𝓕 → (Σ i, f i)
  𝓑p : 𝓕 → (Σ i, f i)
  𝓧n : 𝓕 → ℂ
  𝓧p : 𝓕 → ℂ
  h𝓑 : ∀ i, ofListLift f [i] 1 = ofList [𝓑n i] (𝓧n i) + ofList [𝓑p i] (𝓧p i)
  h𝓑n : ∀ i j, le1 (𝓑n i) j
  h𝓑p : ∀ i j, le1 j (𝓑p i)
```

def toCenterTerm [source]

In the static wick's theorem, the term associated with a contraction `c` containing the contractions.

```
noncomputable def toCenterTerm (f : 𝓕 → Type) [∀ i, Fintype (f i)]
    (q : 𝓕 → FieldStatistic)
    (le1 : (Σ i, f i) → (Σ i, f i) → Prop) [DecidableRel le1]
    {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ (Σ i, f i) →ₐ[ℂ] A) :
    {r : List 𝓕} → (c : Contractions r) → (S : Splitting f le1) → A
  | [], ⟨[], .nil⟩, _ => 1
  | _ :: _, ⟨_, .cons (aux := aux') none c⟩, S => toCenterTerm f q le1 F ⟨aux', c⟩ S
  | a :: _, ⟨_, .cons (aux := aux') (some n) c⟩, S => toCenterTerm f q le1 F ⟨aux', c⟩ S *
    superCommuteCoef q [aux'.get n] (List.take (↑n) aux') •
      F (((superCommute fun i => q i.fst) (ofList [S.𝓑p a] (S.𝓧p a))) (ofListLift f [aux'.get
    n] 1))
```

lemma toCenterTerm_none [source]

Shows that adding a field with no contractions (none) to an existing set of contractions results in the same center term as the original contractions.

Physically, this represents that an uncontracted (free) field does not affect the contraction structure of other fields in Wick's theorem.

```
lemma toCenterTerm_none (f : 𝓕 → Type) [∀ i, Fintype (f i)]
    (q : 𝓕 → FieldStatistic) {r : List 𝓕}
    (le1 : (Σ i, f i) → (Σ i, f i) → Prop) [DecidableRel le1]
```

```
    {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ (Σ i, f i) →ₐ A)
    (S : Splitting f le1) (a : 𝓕) (c : Contractions r) :
    toCenterTerm (r := a :: r) f q le1 F (Contractions.consEquiv.symm (c, none)) S =
    toCenterTerm f q le1 F c S
```

lemma toCenterTerm_center [source]

Proves that the part of the term gained from Wick contractions is in the center of the algebra.

```
lemma toCenterTerm_center (f : 𝓕 → Type) [∀ i, Fintype (f i)]
    (q : 𝓕 → FieldStatistic)
    (le : (Σ i, f i) → (Σ i, f i) → Prop) [DecidableRel le]
    {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ (Σ i, f i) →ₐ A) [OperatorMap (fun i => q i.1) le F] :
    {r : List 𝓕} → (c : Contractions r) → (S : Splitting f le) →
    (c.toCenterTerm f q le F S) ∈ Subalgebra.center ℂ A
  | [], ⟨[], .nil⟩, _ => by
    dsimp [toCenterTerm]
    exact Subalgebra.one_mem (Subalgebra.center ℂ A)
  | _ :: _, ⟨_, .cons (aux := aux') none c⟩, S => by
    dsimp [toCenterTerm]
    exact toCenterTerm_center f q le F ⟨aux', c⟩ S
  | a :: _, ⟨_, .cons (aux := aux') (some n) c⟩, S => by
    dsimp [toCenterTerm]
    refine Subalgebra.mul_mem (Subalgebra.center ℂ A) ?hx ?hy
    exact toCenterTerm_center f q le F ⟨aux', c⟩ S
    apply Subalgebra.smul_mem
    rw [ofListLift_expand]
    rw [map_sum, map_sum]
    refine Subalgebra.sum_mem (Subalgebra.center ℂ A) ?hy.hx.h
    intro x _
    simp only [CreateAnnihilateSect.toList]
    rw [ofList_singleton]
    exact OperatorMap.superCommute_ofList_singleton_ι_center (q := fun i => q i.1)
      (le := le) F (S.ℬp a) ⟨aux'[↑n], x.head⟩
```

2.11 HepLean.PerturbationTheory.Contractions.Involutions

HepLean/PerturbationTheory/Contractions/Involutions.lean on GitHub.

informal_definition equivInvolution [source]

```
informal_definition equivInvolution where
  math :≈ "There is an isomorphism between the type of contractions of a list `l` and
    the type of involutions from `Fin l.length` to `Fin l.length."
```

informal_definition equivFullInvolution [source]

```
informal_definition equivFullInvolution where
  math :≈ "There is an isomorphism from the type of full contractions of a list `l`
    and the type of fixed-point free involutions from `Fin l.length` to `Fin l.length."
```

2.12 HepLean.PerturbationTheory.FieldStatistics

HepLean/PerturbationTheory/FieldStatistics.lean on GitHub.

inductive FieldStatistic [source]

A field can either be bosonic or fermionic in nature. That is to say, they can either have Bose-Einstein statistics or Fermi-Dirac statistics.

```
inductive FieldStatistic : Type where
  | bosonic : FieldStatistic
  | fermionic : FieldStatistic
```

instance Fintype FieldStatistic [source]

Field statics form a finite type.

```
instance : Fintype FieldStatistic where
```

lemma fermionic_not_eq_bonsic [source]

```
@[simp]
lemma fermionic_not_eq_bonsic : ¬ fermionic = bosonic
```

lemma bonsic_eq_fermionic_false [source]

```
lemma bonsic_eq_fermionic_false : bosonic = fermionic ↔ false
```

lemma neq_fermionic_iff_eq_bosonic [source]

```
@[simp]
lemma neq_fermionic_iff_eq_bosonic (a : FieldStatistic) : ¬ a = fermionic ↔ a = bosonic
```

lemma bosonic_neq_iff_fermionic_eq [source]

```
@[simp]
lemma bosonic_neq_iff_fermionic_eq (a : FieldStatistic) : ¬ bosonic = a ↔ fermionic = a
```

lemma fermionic_neq_iff_bosonic_eq [source]

```
@[simp]
lemma fermionic_neq_iff_bosonic_eq (a : FieldStatistic) : ¬ fermionic = a ↔ bosonic = a
```

lemma eq_self_if_eq_bosonic [source]

```
lemma eq_self_if_eq_bosonic {a : FieldStatistic} :
    (if a = bosonic then bosonic else fermionic) = a
```

lemma eq_self_if_bosonic_eq [source]

```
lemma eq_self_if_bosonic_eq {a : FieldStatistic} :
    (if bosonic = a then bosonic else fermionic) = a
```

def ofList [source]

The field statistics of a list of fields is fermionic if ther is an odd number of fermions, otherwise it is bosonic.

```
def ofList (s : 𝓕 → FieldStatistic) : (φs : List 𝓕) → FieldStatistic
  | [] => bosonic
  | φ :: φs => if s φ = ofList s φs then bosonic else fermionic
```

lemma ofList_singleton [source]

```
@[simp]
lemma ofList_singleton (s : 𝓕 → FieldStatistic) (φ : 𝓕) : ofList s [φ] = s φ
```

lemma ofList_freeMonoid [source]

```
@[simp]
lemma ofList_freeMonoid (s : 𝓕 → FieldStatistic) (φ : 𝓕) : ofList s (FreeMonoid.of φ) = s φ
```

lemma ofList_empty [source]

```
@[simp]
lemma ofList_empty (s : 𝓕 → FieldStatistic) : ofList s [] = bosonic
```

lemma ofList_append [source]

```
@[simp]
lemma ofList_append (s : 𝓕 → FieldStatistic) (l r : List 𝓕) :
    ofList s (l ++ r) = if ofList s l = ofList s r then bosonic else fermionic
```

lemma ofList_eq_countP [source]

```
lemma ofList_eq_countP (s : 𝓕 → FieldStatistic) (φs : List 𝓕) :
    ofList s φs = if Nat.mod (List.countP (fun i => decide (s i = fermionic)) φs) 2 = 0 then
      bosonic else fermionic
```

lemma ofList_perm [source]

```
lemma ofList_perm (s : 𝓕 → FieldStatistic) {l l' : List 𝓕} (h : l.Perm l') :
    ofList s l = ofList s l'
```

lemma ofList_orderedInsert [source]

```
lemma ofList_orderedInsert (s : 𝓕 → FieldStatistic) (le1 : 𝓕 → 𝓕 → Prop) [DecidableRel le1]
    (φs : List 𝓕) (φ : 𝓕) : ofList s (List.orderedInsert le1 φ φs) = ofList s (φ :: φs)
```

lemma ofList_insertionSort [source]

```
@[simp]
lemma ofList_insertionSort (s : 𝓕 → FieldStatistic) (le1 : 𝓕 → 𝓕 → Prop) [DecidableRel le1]
    (φs : List 𝓕) : ofList s (List.insertionSort le1 φs) = ofList s φs
```

2.13 HepLean.PerturbationTheory.Wick.CreateAnnihilateSection

HepLean/PerturbationTheory/Wick/CreateAnnihilateSection.lean on GitHub.

def CreateAnnihilateSect [source]

The sections of `Σ i, f i` over a list `l : List 𝓕`. In terms of physics, given some fields $\varphi_1 ... \varphi_n$, the different ways one can associate each field as a `creation` or an `annilation` operator. E.g. the number of terms $\varphi_1^0 \varphi_2^1 ... \varphi_n^0$ $\varphi_1^1 \varphi_2^1 ... \varphi_n^0$ etc. If some fields are exclusively creation or annhilation operators at this point (e.g. ansymptotic states) this is accounted for.

```
def CreateAnnihilateSect {𝓕 : Type} (f : 𝓕 → Type) (l : List 𝓕) : Type
```

instance fintype [source]

The type `CreateAnnihilateSect f l` is finite.

```
instance fintype : Fintype (CreateAnnihilateSect f l)
```

def tail [source]

The section got by dropping the first element of `l` if it exists.

```
def tail : {l : List 𝓕} → (a : CreateAnnihilateSect f l) → CreateAnnihilateSect f l.tail
  | [], a => a
  | _ :: _, a => fun i => a (Fin.succ i)
```

def head [source]

For a list of fields `i :: l` the value of the section at the head `i`.

```
def head {i : 𝓕} (a : CreateAnnihilateSect f (i :: l)) : f i
```

def toList [source]

The list `List (Σ i, f i)` defined by `a`.

```
def toList : {l : List 𝓕} → (a : CreateAnnihilateSect f l) → List (Σ i, f i)
  | [], _ => []
  | i :: _, a => ⟨i, a.head⟩ :: toList a.tail
```

lemma toList_length [source]

```
@[simp]
lemma toList_length : (toList a).length = l.length
```

lemma toList_tail [source]

```
lemma toList_tail : {l : List 𝓕} → (a : CreateAnnihilateSect f l) → toList a.tail = (toList a).
    tail
  | [], _ => rfl
  | i :: l, a => by
    simp [toList]
```

lemma toList_cons [source]

```
lemma toList_cons {i : 𝓕} (a : CreateAnnihilateSect f (i :: l)) :
    (toList a) = ⟨i, a.head⟩ :: toList a.tail
```

lemma toList_get [source]

```
lemma toList_get (a : CreateAnnihilateSect f l) :
    (toList a).get = (fun i => ⟨l.get i, a i⟩) ∘ Fin.cast (by simp)
```

lemma toList_grade [source]

```
@[simp]
lemma toList_grade :
    FieldStatistic.ofList (fun i => q i.fst) a.toList = fermionic ↔
    FieldStatistic.ofList q l = fermionic
```

lemma toList_grade_take [source]

```
@[simp]
lemma toList_grade_take (q : 𝓕 → FieldStatistic) :
    (r : List 𝓕) → (a : CreateAnnihilateSect f r) → (n : ℕ) →
    ofList (fun i => q i.fst) (List.take n a.toList) = ofList q (List.take n r)
  | [], _, _ => by
    simp [toList]
  | i :: r, a, 0 => by
    simp
  | i :: r, a, Nat.succ n => by
    simp only [ofList, Fin.isValue]
    rw [toList_grade_take q r a.tail n]
```

def extractEquiv [source]

The equivalence between `CreateAnnihilateSect f l` and `f (l.get n) × CreateAnnihilateSect f (l.eraseIdx n)` obtained by extracting the `n`th field from `l`.

```
def extractEquiv (n : Fin l.length) : CreateAnnihilateSect f l ≃
    f (l.get n) × CreateAnnihilateSect f (l.eraseIdx n)
```

lemma extractEquiv_symm_toList_get_same [source]

```
lemma extractEquiv_symm_toList_get_same (n : Fin l.length) (a0 : f (l.get n))
    (a : CreateAnnihilateSect f (l.eraseIdx n)) :
    ((extractEquiv n).symm (a0, a)).toList[n] = ⟨l[n], a0⟩
```

def eraseIdx [source]

The section obtained by dropping the `n`th field.

```
def eraseIdx (a : CreateAnnihilateSect f l) (n : Fin l.length) :
    CreateAnnihilateSect f (l.eraseIdx n)
```

lemma eraseIdx_zero_tail [source]

```
@[simp]
lemma eraseIdx_zero_tail {i : 𝓕} (a : CreateAnnihilateSect f (i :: l)) :
    (eraseIdx a (@OfNat.ofNat (Fin (l.length + 1)) 0 Fin.instOfNat : Fin (l.length + 1))) =
    a.tail
```

lemma eraseIdx_succ_head [source]

```
lemma eraseIdx_succ_head {i : 𝓕} (n : ℕ) (hn : n + 1 < (i :: l).length)
    (a : CreateAnnihilateSect f (i :: l)) : (eraseIdx a ⟨n + 1, hn⟩).head = a.head
```

lemma eraseIdx_succ_tail [source]

```
lemma eraseIdx_succ_tail {i : 𝓕} (n : ℕ) (hn : n + 1 < (i :: l).length)
    (a : CreateAnnihilateSect f (i :: l)) :
    (eraseIdx a ⟨n + 1, hn⟩).tail = eraseIdx a.tail ⟨n, Nat.succ_lt_succ_iff.mp hn⟩
```

lemma eraseIdx_toList [source]

```
lemma eraseIdx_toList : {l : List 𝓕} → {n : Fin l.length} → (a : CreateAnnihilateSect f l) →
    (eraseIdx a n).toList = a.toList.eraseIdx n
  | [], n, _ => Fin.elim0 n
  | r0 :: r, ⟨0, h⟩, a => by
    simp [toList_tail]
  | r0 :: r, ⟨n + 1, h⟩, a => by
    simp only [toList, List.length_cons, List.tail_cons, List.eraseIdx_cons_succ, List.cons.
    injEq,
      Sigma.mk.inj_iff, heq_eq_eq, true_and]
    apply And.intro
    · rw [eraseIdx_succ_head]
    · conv_rhs => rw [← eraseIdx_toList (l := r) (n := ⟨n, Nat.succ_lt_succ_iff.mp h⟩) a.tail]
      rw [eraseIdx_succ_tail]
```

lemma extractEquiv_symm_eraseIdx [source]

```
lemma extractEquiv_symm_eraseIdx {I : Type} {f : I → Type}
    {l : List I} (n : Fin l.length) (a0 : f l[↑n]) (a : CreateAnnihilateSect f (l.eraseIdx n))
    :
    ((extractEquiv n).symm (a0, a)).eraseIdx n = a
```

lemma toList_koszulSignInsert [source]

```
lemma toList_koszulSignInsert (x : (i : 𝓕) × f i) :
    koszulSignInsert (fun i => q i.fst) (fun i j => le i.fst j.fst) x a.toList =
    koszulSignInsert q le x.1 l
```

lemma toList_koszulSign [source]

```
lemma toList_koszulSign :
    koszulSign (fun i => q i.fst) (fun i j => le i.fst j.fst) a.toList =
    koszulSign q le l
```

lemma insertionSortEquiv_toList [source]

```
lemma insertionSortEquiv_toList :
    insertionSortEquiv (fun i j => le i.fst j.fst) a.toList =
    (Fin.castOrderIso (by simp)).toEquiv.trans ((insertionSortEquiv le l).trans
    (Fin.castOrderIso (by simp)).toEquiv)
```

def sort [source]

Given a section for `l` the corresponding section for `List.insertionSort le1 l`.

```
def sort :
    CreateAnnihilateSect f (List.insertionSort le l)
```

lemma sort_toList [source]

```
lemma sort_toList :
    (a.sort le).toList = List.insertionSort (fun i j => le i.fst j.fst) a.toList
```

2.14 HepLean.PerturbationTheory.Wick.KoszulOrder

HepLean/PerturbationTheory/Wick/KoszulOrder.lean on GitHub.

def koszulOrderMonoidAlgebra [source]

Given a relation `r` on `I` sorts elements of `MonoidAlgebra ℂ (FreeMonoid I)` by `r` giving it a signed dependent on `q`.

```
def koszulOrderMonoidAlgebra :
    MonoidAlgebra ℂ (FreeMonoid 𝓕) →ₗ[ℂ] MonoidAlgebra ℂ (FreeMonoid 𝓕)
```

lemma koszulOrderMonoidAlgebra_ofList [source]

```
lemma koszulOrderMonoidAlgebra_ofList (i : List 𝓕) :
    koszulOrderMonoidAlgebra q le (MonoidAlgebra.of ℂ (FreeMonoid 𝓕) i) =
    (koszulSign q le i) • (MonoidAlgebra.of ℂ (FreeMonoid 𝓕) (List.insertionSort le i))
```

lemma koszulOrderMonoidAlgebra_single [source]

```
@[simp]
lemma koszulOrderMonoidAlgebra_single (l : FreeMonoid 𝓕) (x : ℂ) :
    koszulOrderMonoidAlgebra q le (MonoidAlgebra.single l x)
    = (koszulSign q le l) • (MonoidAlgebra.single (List.insertionSort le l) x)
```

def koszulOrder [source]

Given a relation `r` on `I` sorts elements of `FreeAlgebra ℂ I` by `r` giving it a signed dependent on `q`.

```
def koszulOrder : FreeAlgebra ℂ 𝓕 →ₗ[ℂ] FreeAlgebra ℂ 𝓕
```

lemma koszulOrder_ι [source]

```
@[simp]
lemma koszulOrder_ι (i : 𝓕) : koszulOrder q le (FreeAlgebra.ι ℂ i) = FreeAlgebra.ι ℂ i
```

lemma koszulOrder_single [source]

```
@[simp]
lemma koszulOrder_single (l : FreeMonoid 𝓕) :
    koszulOrder q le (FreeAlgebra.equivMonoidAlgebraFreeMonoid.symm (MonoidAlgebra.single l x))
    = FreeAlgebra.equivMonoidAlgebraFreeMonoid.symm
    (MonoidAlgebra.single (List.insertionSort le l) (koszulSign q le l * x))
```

lemma koszulOrder_ι_pair [source]

```
@[simp]
lemma koszulOrder_ι_pair (i j : 𝓕) : koszulOrder q le (FreeAlgebra.ι ℂ i * FreeAlgebra.ι ℂ j)
    =
    if le i j then FreeAlgebra.ι ℂ i * FreeAlgebra.ι ℂ j else
    (koszulSign q le [i, j]) • (FreeAlgebra.ι ℂ j * FreeAlgebra.ι ℂ i)
```

lemma koszulOrder_one [source]

```
@[simp]
lemma koszulOrder_one : koszulOrder q le 1 = 1
```

lemma mul_koszulOrder_le [source]

```
lemma mul_koszulOrder_le (i : 𝓕) (A : FreeAlgebra ℂ 𝓕) (hi : ∀ j, le i j) :
    FreeAlgebra.ι ℂ i * koszulOrder q le A = koszulOrder q le (FreeAlgebra.ι ℂ i * A)
```

lemma koszulOrder_mul_ge [source]

```
lemma koszulOrder_mul_ge (i : 𝓕) (A : FreeAlgebra ℂ 𝓕) (hi : ∀ j, le j i) :
    koszulOrder q le A * FreeAlgebra.ι ℂ i = koszulOrder q le (A * FreeAlgebra.ι ℂ i)
```

2.15 HepLean.PerturbationTheory.Wick.OfList

HepLean/PerturbationTheory/Wick/OfList.lean on GitHub.

def ofList [source]

The element of the free algebra `FreeAlgebra ℂ I` associated with a `List I`.

```
def ofList (l : List 𝓕) (x : ℂ) : FreeAlgebra ℂ 𝓕
```

lemma ofList_pair [source]

```
lemma ofList_pair (l r : List 𝓕) (x y : ℂ) :
    ofList (l ++ r) (x * y) = ofList l x * ofList r y
```

lemma ofList_triple [source]

```
lemma ofList_triple (la lb lc : List 𝓕) (xa xb xc : ℂ) :
    ofList (la ++ lb ++ lc) (xa * xb * xc) = ofList la xa * ofList lb xb * ofList lc xc
```

lemma ofList_triple_assoc [source]

```
lemma ofList_triple_assoc (la lb lc : List 𝓕) (xa xb xc : ℂ) :
    ofList (la ++ (lb ++ lc)) (xa * (xb * xc)) = ofList la xa * ofList lb xb * ofList lc xc
```

lemma ofList_cons_eq_ofList [source]

```
lemma ofList_cons_eq_ofList (l : List 𝓕) (i : 𝓕) (x : ℂ) :
    ofList (i :: l) x = ofList [i] 1 * ofList l x
```

lemma ofList_singleton [source]

```
lemma ofList_singleton (i : 𝓕) :
    ofList [i] 1 = FreeAlgebra.ι ℂ i
```

lemma ofList_eq_smul_one [source]

```
lemma ofList_eq_smul_one (l : List 𝓕) (x : ℂ) :
    ofList l x = x • ofList l 1
```

lemma ofList_empty [source]

```
lemma ofList_empty : ofList [] 1 = (1 : FreeAlgebra ℂ 𝓕)
```

lemma ofList_empty' [source]

```
lemma ofList_empty' : ofList [] x = x • (1 : FreeAlgebra ℂ 𝓕)
```

lemma koszulOrder_ofList [source]

```
lemma koszulOrder_ofList
    (l : List 𝓕) (x : ℂ) :
    koszulOrder q le (ofList l x) = (koszulSign q le l) • ofList (List.insertionSort le l) x
```

lemma ofList_insertionSort_eq_koszulOrder [source]

```
lemma ofList_insertionSort_eq_koszulOrder (l : List 𝓕) (x : ℂ) :
    ofList (List.insertionSort le l) x = (koszulSign q le l) • koszulOrder q le (ofList l x)
```

def sumFiber [source]

The map of algebras from `FreeAlgebra ℂ I` to `FreeAlgebra ℂ (Σ i, f i)` taking the monomial `i` to the sum of elements in `(Σ i, f i)` above `i`, i.e. the sum of the fiber above `i`.

```
def sumFiber (f : 𝓕 → Type) [∀ i, Fintype (f i)] :
    FreeAlgebra ℂ 𝓕 →ₐ[ℂ] FreeAlgebra ℂ (Σ i, f i)
```

lemma sumFiber_ι [source]

```
lemma sumFiber_ι (f : 𝓕 → Type) [∀ i, Fintype (f i)] (i : 𝓕) :
    sumFiber f (FreeAlgebra.ι ℂ i) = ∑ (b : f i), FreeAlgebra.ι ℂ ⟨i, b⟩
```

def ofListLift [source]

Given a list `l : List I` the corresponding element of `FreeAlgebra ℂ (Σ i, f i)` by mapping each `i : I` to the sum of it's fiber in `Σ i, f i` and taking the product of the result. For example, in terms of creation and annihlation opperators, `[φ₁, φ₂, φ₃]` gets taken to `(φ₁⁰ + φ₁¹)(φ₂⁰ + φ₂¹)(φ₃⁰ + φ₃¹)`.

```
def ofListLift (f : 𝓕 → Type) [∀ i, Fintype (f i)] (l : List 𝓕) (x : ℂ) :
    FreeAlgebra ℂ (Σ i, f i)
```

lemma ofListLift_empty [source]

```
lemma ofListLift_empty (f : 𝓕 → Type) [∀ i, Fintype (f i)] :
    ofListLift f [] 1 = 1
```

lemma ofListLift_empty_smul [source]

```
lemma ofListLift_empty_smul (f : 𝓕 → Type) [∀ i, Fintype (f i)] (x : ℂ) :
    ofListLift f [] x = x • 1
```

lemma ofListLift_cons [source]

```
lemma ofListLift_cons (f : 𝓕 → Type) [∀ i, Fintype (f i)] (i : 𝓕) (r : List 𝓕) (x : ℂ) :
    ofListLift f (i :: r) x = (∑ j : f i, FreeAlgebra.ι ℂ ⟨i, j⟩) * (ofListLift f r x)
```

lemma ofListLift_singleton [source]

```
lemma ofListLift_singleton (f : 𝓕 → Type) [∀ i, Fintype (f i)] (i : 𝓕) (x : ℂ) :
    ofListLift f [i] x = ∑ j : f i, x • FreeAlgebra.ι ℂ ⟨i, j⟩
```

lemma ofListLift_singleton_one [source]

```
lemma ofListLift_singleton_one (f : 𝓕 → Type) [∀ i, Fintype (f i)] (i : 𝓕) :
    ofListLift f [i] 1 = ∑ j : f i, FreeAlgebra.ι ℂ ⟨i, j⟩
```

lemma ofListLift_cons_eq_ofListLift [source]

```
lemma ofListLift_cons_eq_ofListLift (f : 𝓕 → Type) [∀ i, Fintype (f i)] (i : 𝓕)
    (r : List 𝓕) (x : ℂ) :
    ofListLift f (i :: r) x = ofListLift f [i] 1 * ofListLift f r x
```

lemma ofListLift_expand [source]

```
lemma ofListLift_expand (f : 𝓕 → Type) [∀ i, Fintype (f i)] (x : ℂ) :
    (l : List 𝓕) → ofListLift f l x = ∑ (a : CreateAnnihilateSect f l), ofList a.toList x
  | [] => by
    simp only [ofListLift, CreateAnnihilateSect, List.length_nil, List.get_eq_getElem,
```

```
    Finset.univ_unique, CreateAnnihilateSect.toList, Finset.sum_const, Finset.card_singleton,
    one_smul]
  rw [ofList_eq_smul_one, map_smul, ofList_empty, ofList_eq_smul_one, ofList_empty, map_one]
| i :: l => by
  rw [ofListLift_cons, ofListLift_expand f x l]
  conv_rhs => rw [← (CreateAnnihilateSect.extractEquiv
      ⟨0, by exact Nat.zero_lt_succ l.length⟩).symm.sum_comp (α := FreeAlgebra ℂ _)]
  erw [Finset.sum_product]
  rw [Finset.sum_mul]
  conv_lhs =>
    rhs
    intro n
    rw [Finset.mul_sum]
  congr
  funext j
  congr
  funext n
  rw [← ofList_singleton, ← ofList_pair, one_mul]
  rfl
```

lemma koszulOrder_ofListLift [source]

```
lemma koszulOrder_ofListLift {f : 𝓕 → Type} [∀ i, Fintype (f i)]
    (l : List 𝓕) (x : ℂ) :
    koszulOrder (fun i => q i.fst) (fun i j => le i.1 j.1) (ofListLift f l x) =
    sumFiber f (koszulOrder q le (ofList l x))
```

lemma koszulOrder_ofListLift_eq_ofListLift [source]

```
lemma koszulOrder_ofListLift_eq_ofListLift {f : 𝓕 → Type} [∀ i, Fintype (f i)]
    (l : List 𝓕) (x : ℂ) : koszulOrder (fun i => q i.fst) (fun i j => le i.1 j.1)
    (ofListLift f l x) =
    koszulSign q le l • ofListLift f (List.insertionSort le l) x
```

2.16 HepLean.PerturbationTheory.Wick.OperatorMap

HepLean/PerturbationTheory/Wick/OperatorMap.lean on GitHub.

class OperatorMap [source]

A map from the free algebra of fields `FreeAlgebra ℂ 𝓕` to an algebra `A`, to be thought of as the operator algebra is said to be an operator map if all super commutors of fields land in the center of `A`, if two fields are of a different grade then their super commutor lands on zero, and the `koszulOrder` (normal order) of any term with a super commutor of two fields present is zero. This can be thought as as a condtion on the operator algebra `A` as much as it can on `F`.

```
class OperatorMap {A : Type} [Semiring A] [Algebra ℂ A]
    (q : 𝓕 → FieldStatistic) (le : 𝓕 → 𝓕 → Prop)
    [DecidableRel le] (F : FreeAlgebra ℂ 𝓕 →ₐ[ℂ] A) : Prop where
  superCommute_mem_center : ∀ i j, F (superCommute q (FreeAlgebra.ι ℂ i) (FreeAlgebra.ι ℂ j)) ∈
    Subalgebra.center ℂ A
  superCommute_diff_grade_zero : ∀ i j, q i ≠ q j →
    F (superCommute q (FreeAlgebra.ι ℂ i) (FreeAlgebra.ι ℂ j)) = 0
  superCommute_ordered_zero : ∀ i j, ∀ a b,
```

```
    F (koszulOrder q le (a * superCommute q (FreeAlgebra.ι ℂ i) (FreeAlgebra.ι ℂ j) * b)) = 0
```

lemma superCommute_ofList_singleton_ι_center [source]

```
lemma superCommute_ofList_singleton_ι_center [OperatorMap q le F] (i j : 𝓕) :
    F (superCommute q (ofList [i] xa) (FreeAlgebra.ι ℂ j)) ∈ Subalgebra.center ℂ A
```

lemma superCommuteSplit_operatorMap [source]

```
lemma superCommuteSplit_operatorMap (lb : List 𝓕) (xa xb : ℂ) (n : ℕ)
    (hn : n < lb.length) {A : Type} [Semiring A] [Algebra ℂ A] (f : FreeAlgebra ℂ 𝓕 →ₐ[ℂ] A)
    [OperatorMap q le f] (i : 𝓕) :
    f (superCommuteSplit q [i] lb xa xb n hn) =
    f (superCommute q (ofList [i] xa) (FreeAlgebra.ι ℂ (lb.get ⟨n, hn⟩)))
    * (superCommuteCoef q [i] (List.take n lb) •
    f (ofList (List.eraseIdx lb n) xb))
```

lemma superCommuteLiftSplit_operatorMap [source]

```
lemma superCommuteLiftSplit_operatorMap {f : 𝓕 → Type} [∀ i, Fintype (f i)]
    (c : (Σ i, f i)) (r : List 𝓕) (x y : ℂ) (n : ℕ)
    (hn : n < r.length)
    (le : (Σ i, f i) → (Σ i, f i) → Prop) [DecidableRel le]
    {A : Type} [Semiring A] [Algebra ℂ A] (F : FreeAlgebra ℂ (Σ i, f i) →ₐ[ℂ] A)
    [OperatorMap (fun i => q i.1) le F] :
    F (superCommuteLiftSplit q [c] r x y n hn) = superCommuteLiftCoef q [c] (List.take n r) •
    (F (superCommute (fun i => q i.1) (ofList [c] x)
    (sumFiber f (FreeAlgebra.ι ℂ (r.get ⟨n, hn⟩)))))
    * F (ofListLift f (List.eraseIdx r n) y))
```

lemma superCommute_koszulOrder_le_ofList [source]

```
lemma superCommute_koszulOrder_le_ofList [IsTotal 𝓕 le] [IsTrans 𝓕 le] (r : List 𝓕) (x : ℂ)
    (i : 𝓕) {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ 𝓕 →ₐ A) [OperatorMap q le F] :
    F ((superCommute q (FreeAlgebra.ι ℂ i) (koszulOrder q le (ofList r x)))) =
    ∑ n : Fin r.length, (superCommuteCoef q [r.get n] (r.take n)) •
    (F (((superCommute q) (ofList [i] 1)) (FreeAlgebra.ι ℂ (r.get n))) *
    F ((koszulOrder q le) (ofList (r.eraseIdx ↑n) x)))
```

lemma koszulOrder_of_le_all_ofList [source]

```
lemma koszulOrder_of_le_all_ofList (r : List 𝓕) (x : ℂ) (i : 𝓕)
    {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ 𝓕 →ₐ A) [OperatorMap q le F] :
    F (koszulOrder q le (ofList r x * FreeAlgebra.ι ℂ i))
    = superCommuteCoef q [i] r • F (koszulOrder q le (FreeAlgebra.ι ℂ i * ofList r x))
```

lemma le_all_mul_koszulOrder_ofList [source]

```
lemma le_all_mul_koszulOrder_ofList (r : List 𝓕) (x : ℂ)
    (i : 𝓕) (hi : ∀ (j : 𝓕), le j i)
    {A : Type} [Semiring A] [Algebra ℂ A]
```

```
    (F : FreeAlgebra ℂ 𝓕 →ₐ A) [OperatorMap q le F] :
    F (FreeAlgebra.ι ℂ i * koszulOrder q le (ofList r x)) =
    F ((koszulOrder q le) (FreeAlgebra.ι ℂ i * ofList r x)) +
    F (((superCommute q) (ofList [i] 1)) ((koszulOrder q le) (ofList r x)))
```

def superCommuteCenterOrder [source]

In the expansions of `F (FreeAlgebra.ι ℂ i * koszulOrder q le (ofList r x))` the ter multiplying `F ((koszulOrder q le) (ofList (optionEraseZ r i n) x))`.

```
def superCommuteCenterOrder (r : List 𝓕) (i : 𝓕)
    {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ 𝓕 →ₐ[ℂ] A)
    (n : Option (Fin r.length)) : A
```

lemma superCommuteCenterOrder_none [source]

```
@[simp]
lemma superCommuteCenterOrder_none (r : List 𝓕) (i : 𝓕)
    {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ 𝓕 →ₐ[ℂ] A) :
    superCommuteCenterOrder q r i F none = 1
```

lemma le_all_mul_koszulOrder_ofList_expand [source]

```
lemma le_all_mul_koszulOrder_ofList_expand [IsTotal 𝓕 le] [IsTrans 𝓕 le] (r : List 𝓕) (x : ℂ)
    (i : 𝓕) (hi : ∀ (j : 𝓕), le j i)
    {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ 𝓕 →ₐ[ℂ] A) [OperatorMap q le F] :
    F (FreeAlgebra.ι ℂ i * koszulOrder q le (ofList r x)) =
    ∑ n, superCommuteCenterOrder q r i F n *
    F ((koszulOrder q le) (ofList (optionEraseZ r i n) x))
```

lemma le_all_mul_koszulOrder_ofListLift_expand [source]

```
lemma le_all_mul_koszulOrder_ofListLift_expand {f : 𝓕 → Type} [∀ i, Fintype (f i)]
    (r : List 𝓕) (x : ℂ)
    (le : (Σ i, f i) → (Σ i, f i) → Prop) [DecidableRel le]
    [IsTotal (Σ i, f i) le] [IsTrans (Σ i, f i) le]
    (i : (Σ i, f i)) (hi : ∀ (j : (Σ i, f i)), le j i)
    {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ (Σ i, f i) →ₐ A) [OperatorMap (fun i => q i.1) le F] :
    F (ofList [i] 1 * koszulOrder (fun i => q i.1) le (ofListLift f r x)) =
    F ((koszulOrder (fun i => q i.fst) le) (ofList [i] 1 * ofListLift f r x)) +
    ∑ n : (Fin r.length), superCommuteCoef q [r.get n] (List.take (↑n) r) •
      F (((superCommute fun i => q i.fst) (ofList [i] 1)) (ofListLift f [r.get n] 1)) *
    F ((koszulOrder (fun i => q i.fst) le) (ofListLift f (r.eraseIdx ↑n) x))
```

2.17 HepLean.PerturbationTheory.Wick.Signs.InsertSign

HepLean/PerturbationTheory/Wick/Signs/InsertSign.lean on GitHub.

lemma take_insert_same [source]

```
lemma take_insert_same {I : Type} (i : I) :
    (n : ℕ) → (r : List I) →
    List.take n (List.insertIdx n i r) = List.take n r
  | 0, _ => by simp
  | _+1, [] => by simp
  | n+1, a::as => by
    simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.cons.injEq, true_and]
    exact take_insert_same i n as
```

lemma take_eraseIdx_same [source]

```
lemma take_eraseIdx_same {I : Type} :
    (n : ℕ) → (r : List I) →
    List.take n (List.eraseIdx r n) = List.take n r
  | 0, _ => by simp
  | _+1, [] => by simp
  | n+1, a::as => by
    simp only [List.eraseIdx_cons_succ, List.take_succ_cons, List.cons.injEq, true_and]
    exact take_eraseIdx_same n as
```

lemma take_insert_gt [source]

```
lemma take_insert_gt {I : Type} (i : I) :
    (n m : ℕ) → (h : n < m) → (r : List I) →
    List.take n (List.insertIdx m i r) = List.take n r
  | 0, 0, _, _ => by simp
  | 0, m + 1, _, _ => by simp
  | n+1, m + 1, _, [] => by simp
  | n+1, m + 1, h, a::as => by
    simp only [List.insertIdx_succ_cons, List.take_succ_cons, List.cons.injEq, true_and]
    refine take_insert_gt i n m (Nat.succ_lt_succ_iff.mp h) as
```

lemma take_insert_let [source]

```
lemma take_insert_let {I : Type} (i : I) :
    (n m : ℕ) → (h : m ≤ n) → (r : List I) → (hm : m ≤ r.length) →
    (List.take (n + 1) (List.insertIdx m i r)).Perm (i :: List.take n r)
  | 0, 0, h, _, _ => by simp
  | m + 1, 0, h, r, _ => by simp
  | n + 1, m + 1, h, [], hm => by
    simp at hm
  | n + 1, m + 1, h, a::as, hm => by
    simp only [List.insertIdx_succ_cons, List.take_succ_cons]
    have hp : (i :: a :: List.take n as).Perm (a :: i :: List.take n as)
```

def insertSign [source]

The sign associated with inserting `r0` into `r` at the position `n`. That is the sign associated with commuting `r0` with `List.take n r`.

```
def insertSign (n : ℕ) (r0 : 𝓕) (r : List 𝓕) : ℂ
```

lemma insertSign_insert [source]

```
lemma insertSign_insert (n : ℕ) (r0 : 𝓕) (r : List 𝓕) :
    insertSign q n r0 r = insertSign q n r0 (List.insertIdx n r0 r)
```

lemma insertSign_eraseIdx [source]

```
lemma insertSign_eraseIdx (n : ℕ) (r0 : 𝓕) (r : List 𝓕) :
    insertSign q n r0 (r.eraseIdx n) = insertSign q n r0 r
```

lemma insertSign_zero [source]

```
lemma insertSign_zero (r0 : 𝓕) (r : List 𝓕) : insertSign q 0 r0 r = 1
```

lemma insertSign_succ_cons [source]

```
lemma insertSign_succ_cons (n : ℕ) (r0 r1 : 𝓕) (r : List 𝓕) : insertSign q (n + 1) r0 (r1 :: r
    ) =
    superCommuteCoef q [r0] [r1] * insertSign q n r0 r
```

lemma insertSign_insert_gt [source]

```
lemma insertSign_insert_gt (n m : ℕ) (r0 r1 : 𝓕) (r : List 𝓕) (hn : n < m) :
    insertSign q n r0 (List.insertIdx m r1 r) = insertSign q n r0 r
```

lemma insertSign_insert_lt_eq_insertSort [source]

```
lemma insertSign_insert_lt_eq_insertSort (n m : ℕ) (r0 r1 : 𝓕) (r : List 𝓕) (hn : m ≤ n)
    (hm : m ≤ r.length) :
    insertSign q (n + 1) r0 (List.insertIdx m r1 r) = insertSign q (n + 1) r0 (r1 :: r)
```

lemma insertSign_insert_lt [source]

```
lemma insertSign_insert_lt (n m : ℕ) (r0 r1 : 𝓕) (r : List 𝓕) (hn : m ≤ n) (hm : m ≤ r.length)
     :
    insertSign q (n + 1) r0 (List.insertIdx m r1 r) = superCommuteCoef q [r0] [r1] *
    insertSign q n r0 r
```

2.18 HepLean.PerturbationTheory.Wick.Signs.KoszulSign

HepLean/PerturbationTheory/Wick/Signs/KoszulSign.lean on GitHub.

def koszulSign [source]

Gives a factor of `-1` for every fermion-fermion (`q` is `1`) crossing that occurs when sorting a list of based on `r`.

```
def koszulSign (q : 𝓕 → FieldStatistic) (le : 𝓕 → 𝓕 → Prop) [DecidableRel le] :
    List 𝓕 → ℂ
  | [] => 1
  | a :: l => koszulSignInsert q le a l * koszulSign q le l
```

lemma koszulSign_mul_self [source]

```
lemma koszulSign_mul_self (l : List 𝓕) : koszulSign q le l * koszulSign q le l = 1
```

lemma koszulSign_freeMonoid_of [source]

```
@[simp]
lemma koszulSign_freeMonoid_of (i : 𝓕) : koszulSign q le (FreeMonoid.of i) = 1
```

lemma koszulSignInsert_erase_boson [source]

```
lemma koszulSignInsert_erase_boson {𝓕 : Type} (q : 𝓕 → FieldStatistic)
    (le : 𝓕 → 𝓕 → Prop) [DecidableRel le] (r0 : 𝓕) :
    (r : List 𝓕) → (n : Fin r.length) → (heq : q (r.get n) = bosonic) →
    koszulSignInsert q le r0 (r.eraseIdx n) = koszulSignInsert q le r0 r
  | [], _, _ => by
    simp
  | r1 :: r, ⟨0, h⟩, hr => by
    simp only [List.eraseIdx_zero, List.tail_cons]
    simp only [List.length_cons, Fin.zero_eta, List.get_eq_getElem, Fin.val_zero,
      List.getElem_cons_zero] at hr
    rw [koszulSignInsert]
    simp [hr]
  | r1 :: r, ⟨n + 1, h⟩, hr => by
    simp only [List.eraseIdx_cons_succ]
    rw [koszulSignInsert, koszulSignInsert]
    rw [koszulSignInsert_erase_boson q le r0 r ⟨n, Nat.succ_lt_succ_iff.mp h⟩ hr]
```

lemma koszulSign_erase_boson [source]

```
lemma koszulSign_erase_boson {𝓕 : Type} (q : 𝓕 → FieldStatistic) (le : 𝓕 → 𝓕 → Prop)
    [DecidableRel le] :
    (r : List 𝓕) → (n : Fin r.length) → (heq : q (r.get n) = bosonic) →
    koszulSign q le (r.eraseIdx n) = koszulSign q le r
  | [], _ => by
    simp
  | r0 :: r, ⟨0, h⟩ => by
    simp only [List.length_cons, Fin.zero_eta, List.get_eq_getElem, Fin.val_zero,
      List.getElem_cons_zero, Fin.isValue, List.eraseIdx_zero, List.tail_cons, koszulSign]
    intro h
    rw [koszulSignInsert_boson]
    simp only [one_mul]
    exact h
  | r0 :: r, ⟨n + 1, h⟩ => by
    simp only [List.length_cons, List.get_eq_getElem, List.getElem_cons_succ, Fin.isValue,
      List.eraseIdx_cons_succ]
    intro h'
    rw [koszulSign, koszulSign, koszulSign_erase_boson q le r ⟨n, Nat.succ_lt_succ_iff.mp h⟩]
    congr 1
    rw [koszulSignInsert_erase_boson q le r0 r ⟨n, Nat.succ_lt_succ_iff.mp h⟩ h']
    exact h'
```

lemma koszulSign_insertIdx [source]

```
lemma koszulSign_insertIdx [IsTotal 𝓕 le] [IsTrans 𝓕 le] (i : 𝓕) :
```

```
    (r : List 𝓕) → (n : ℕ) → (hn : n ≤ r.length) →
    koszulSign q le (List.insertIdx n i r) = insertSign q n i r * koszulSign q le r *
      insertSign q (insertionSortEquiv le (List.insertIdx n i r) (n, by
        rw [List.length_insertIdx _ _ hn]
        omega)) i (List.insertionSort le (List.insertIdx n i r))
  | [], 0, h => by
    simp [koszulSign, insertSign, superCommuteCoef, koszulSignInsert]
  | [], n + 1, h => by
    simp at h
  | r0 :: r, 0, h => by
    simp only [List.insertIdx_zero, List.insertionSort, List.length_cons, Fin.zero_eta]
    rw [koszulSign]
    trans koszulSign q le (r0 :: r) * koszulSignInsert q le i (r0 :: r)
    ring
    simp only [insertionSortEquiv, List.length_cons, Nat.succ_eq_add_one, List.insertionSort,
      orderedInsertEquiv, OrderIso.toEquiv_symm, Fin.symm_castOrderIso, HepLean.Fin.
    equivCons_trans,
      Equiv.trans_apply, HepLean.Fin.equivCons_zero, HepLean.Fin.finExtractOne_apply_eq,
      Fin.isValue, HepLean.Fin.finExtractOne_symm_inl_apply, RelIso.coe_fn_toEquiv,
      Fin.castOrderIso_apply, Fin.cast_mk, Fin.eta]
    conv_rhs =>
      rhs
      rhs
      rw [orderedInsert_eq_insertIdx_orderedInsertPos]
    conv_rhs =>
      rhs
      rw [← insertSign_insert]
      change insertSign q (↑(orderedInsertPos le ((List.insertionSort le (r0 :: r))) i)) i
        (List.insertionSort le (r0 :: r))
      rw [← koszulSignInsert_eq_insertSign q le]
    rw [insertSign_zero]
    simp
  | r0 :: r, n + 1, h => by
    conv_lhs =>
      rw [List.insertIdx_succ_cons]
      rw [koszulSign]
    rw [koszulSign_insertIdx]
    conv_rhs =>
      rhs
      simp only [List.insertIdx_succ_cons]
```

2.19 HepLean.PerturbationTheory.Wick.Signs.KoszulSignInsert

HepLean/PerturbationTheory/Wick/Signs/KoszulSignInsert.lean on GitHub.

def koszulSignInsert [source]

Gives a factor of `-1` when inserting `a` into a list `List I` in the ordered position for each fermion-fermion cross.

```
def koszulSignInsert {𝓕 : Type} (q : 𝓕 → FieldStatistic) (le : 𝓕 → 𝓕 → Prop)
    [DecidableRel le] (a : 𝓕) : List 𝓕 → ℂ
  | [] => 1
  | b :: l => if le a b then koszulSignInsert q le a l else
    if q a = fermionic ∧ q b = fermionic then - koszulSignInsert q le a l else
      koszulSignInsert q le a l
```

lemma koszulSignInsert_boson [source]

When inserting a boson the `koszulSignInsert` is always `1`.

```
lemma koszulSignInsert_boson (q : 𝓕 → FieldStatistic) (le : 𝓕 → 𝓕 → Prop) [DecidableRel le]
    (a : 𝓕) (ha : q a = bosonic) : (l : List 𝓕) → koszulSignInsert q le a l = 1
  | [] => by
    simp [koszulSignInsert]
  | b :: l => by
    simp only [koszulSignInsert, Fin.isValue, ite_eq_left_iff]
    rw [koszulSignInsert_boson q le a ha l, ha]
    simp only [reduceCtorEq, false_and, ↓reduceIte, ite_self]
```

lemma koszulSignInsert_mul_self [source]

```
@[simp]
lemma koszulSignInsert_mul_self (a : 𝓕) :
    (l : List 𝓕) → koszulSignInsert q le a l * koszulSignInsert q le a l = 1
  | [] => by
    simp [koszulSignInsert]
  | b :: l => by
    simp only [koszulSignInsert, Fin.isValue, mul_ite, ite_mul, neg_mul, mul_neg]
    by_cases hr : le a b
    · simp only [hr, ↓reduceIte]
      rw [koszulSignInsert_mul_self]
    · simp only [hr, ↓reduceIte]
      by_cases hq : q a = fermionic ∧ q b = fermionic
      · simp only [hq, and_self, ↓reduceIte, neg_neg]
        rw [koszulSignInsert_mul_self]
      · simp only [hq, ↓reduceIte]
        rw [koszulSignInsert_mul_self]
```

lemma koszulSignInsert_le_forall [source]

```
lemma koszulSignInsert_le_forall (a : 𝓕) (l : List 𝓕) (hi : ∀ b, le a b) :
    koszulSignInsert q le a l = 1
```

lemma koszulSignInsert_ge_forall_append [source]

```
lemma koszulSignInsert_ge_forall_append (l : List 𝓕) (j i : 𝓕) (hi : ∀ j, le j i) :
    koszulSignInsert q le j l = koszulSignInsert q le j (l ++ [i])
```

lemma koszulSignInsert_eq_filter [source]

```
lemma koszulSignInsert_eq_filter (r0 : 𝓕) : (r : List 𝓕) →
    koszulSignInsert q le r0 r =
    koszulSignInsert q le r0 (List.filter (fun i => decide (¬ le r0 i)) r)
  | [] => by
    simp [koszulSignInsert]
  | r1 :: r => by
    dsimp only [koszulSignInsert, Fin.isValue]
    simp only [Fin.isValue, List.filter, decide_not]
    by_cases h : le r0 r1
    · simp only [h, ↓reduceIte, decide_True, Bool.not_true]
      rw [koszulSignInsert_eq_filter]
      congr
```

```
    simp
  · simp only [h, ↓reduceIte, Fin.isValue, decide_False, Bool.not_false]
    dsimp only [Fin.isValue, koszulSignInsert]
    simp only [Fin.isValue, h, ↓reduceIte]
    rw [koszulSignInsert_eq_filter]
    congr
    simp only [decide_not]
    simp
```

lemma koszulSignInsert_eq_cons [source]

```
lemma koszulSignInsert_eq_cons [IsTotal 𝓕 le] (r0 : 𝓕) (r : List 𝓕) :
    koszulSignInsert q le r0 r = koszulSignInsert q le r0 (r0 :: r)
```

lemma koszulSignInsert_eq_grade [source]

```
lemma koszulSignInsert_eq_grade (r0 : 𝓕) (r : List 𝓕) :
    koszulSignInsert q le r0 r = if ofList q [r0] = fermionic ∧
    ofList q (List.filter (fun i => decide (¬ le r0 i)) r) = fermionic then -1 else 1
```

lemma koszulSignInsert_eq_perm [source]

```
lemma koszulSignInsert_eq_perm (r r' : List 𝓕) (a : 𝓕) (h : r.Perm r') :
    koszulSignInsert q le a r = koszulSignInsert q le a r'
```

lemma koszulSignInsert_eq_sort [source]

```
lemma koszulSignInsert_eq_sort (r : List 𝓕) (a : 𝓕) :
    koszulSignInsert q le a r = koszulSignInsert q le a (List.insertionSort le r)
```

lemma koszulSignInsert_eq_insertSign [source]

```
lemma koszulSignInsert_eq_insertSign [IsTotal 𝓕 le] [IsTrans 𝓕 le] (r0 : 𝓕) (r : List 𝓕) :
    koszulSignInsert q le r0 r = insertSign q (orderedInsertPos le (List.insertionSort le r) r0
    )
      r0 (List.insertionSort le r)
```

lemma koszulSignInsert_insertIdx [source]

```
lemma koszulSignInsert_insertIdx (i j : 𝓕) (r : List 𝓕) (n : ℕ) (hn : n ≤ r.length) :
    koszulSignInsert q le j (List.insertIdx n i r) = koszulSignInsert q le j (i :: r)
```

def koszulSignCons [source]

The difference in `koszulSignInsert` on inserting `r0` into `r` compared to into `r1 :: r` for any `r`.

```
def koszulSignCons (r0 r1 : 𝓕) : ℂ
```

lemma koszulSignCons_eq_superComuteCoef [source]

```
lemma koszulSignCons_eq_superComuteCoef (r0 r1 : 𝓕) : koszulSignCons q le r0 r1 =
    if le r0 r1 then 1 else superCommuteCoef q [r0] [r1]
```

lemma koszulSignInsert_cons [source]

```
lemma koszulSignInsert_cons (r0 r1 : 𝓕) (r : List 𝓕) :
    koszulSignInsert q le r0 (r1 :: r) = (koszulSignCons q le r0 r1) *
    koszulSignInsert q le r0 r
```

2.20 HepLean.PerturbationTheory.Wick.Signs.StaticWickCoef

HepLean/PerturbationTheory/Wick/Signs/StaticWickCoef.lean on GitHub.

def staticWickCoef [source]

The sign that appears in the static version of Wicks theorem. This is actually equal to `superCommuteCoef q [r.get n] (r.take n)`, something which will be proved in a lemma.

```
def staticWickCoef (r : List 𝓕) (i : 𝓕) (n : Fin r.length) : ℂ
```

lemma staticWickCoef_eq_q [source]

```
lemma staticWickCoef_eq_q (r : List 𝓕) (i : 𝓕) (n : Fin r.length)
    (hq : q i = q (r.get n)) :
    staticWickCoef q le r i n =
    koszulSign q le r *
    superCommuteCoef q [r.get n] (List.take (↑(insertionSortEquiv le r n))
      (List.insertionSort le r)) *
    koszulSign q le (r.eraseIdx ↑n)
```

lemma insertIdx_eraseIdx [source]

```
lemma insertIdx_eraseIdx {I : Type} : (n : ℕ) → (r : List I) → (hn : n < r.length) →
    List.insertIdx n (r.get ⟨n, hn⟩) (r.eraseIdx n) = r
  | n, [], hn => by
    simp at hn
  | 0, r0 :: r, hn => by
    simp
  | n + 1, r0 :: r, hn => by
    simp only [List.length_cons, List.get_eq_getElem, List.getElem_cons_succ,
      List.eraseIdx_cons_succ, List.insertIdx_succ_cons, List.cons.injEq, true_and]
    exact insertIdx_eraseIdx n r _
```

lemma staticWickCoef_eq_get [source]

```
lemma staticWickCoef_eq_get [IsTotal 𝓕 le] [IsTrans 𝓕 le] (r : List 𝓕) (i : 𝓕) (n : Fin r.
    length)
    (heq : q i = q (r.get n)) :
    staticWickCoef q le r i n = superCommuteCoef q [r.get n] (r.take n)
```

2.21 HepLean.PerturbationTheory.Wick.Signs.SuperCommuteCoef

HepLean/PerturbationTheory/Wick/Signs/SuperCommuteCoef.lean on GitHub.

def superCommuteCoef [source]

Given two lists `la` and `lb` returns `-1` if they are both of grade `1` and `1` otherwise. This corresponds to the sign associated with the super commutator when commuting `la` and `lb` in the free algebra. In terms of physics it is `-1` if commuting two fermionic operators and `1` otherwise.

```
def superCommuteCoef (la lb : List 𝓕) : ℂ
```

lemma superCommuteCoef_comm [source]

```
lemma superCommuteCoef_comm (la lb : List 𝓕) :
    superCommuteCoef q la lb = superCommuteCoef q lb la
```

def superCommuteLiftCoef [source]

Given a list `l : List (Σ i, f i)` and a list `r : List I` returns `-1` if the grade of `l` is `1` and the grade of `r` is `1` and `1` otherwise. This corresponds to the sign associated with the super commutator when commuting the lift of `l` and `r` (by summing over fibers) in the free algebra over `Σ i, f i`. In terms of physics it is `-1` if commuting two fermionic operators and `1` otherwise.

```
def superCommuteLiftCoef {f : 𝓕 → Type} (l : List (Σ i, f i)) (r : List 𝓕) : ℂ
```

lemma superCommuteLiftCoef_empty [source]

```
lemma superCommuteLiftCoef_empty {f : 𝓕 → Type} (l : List (Σ i, f i)) :
    superCommuteLiftCoef q l [] = 1
```

lemma superCommuteCoef_perm_snd [source]

```
lemma superCommuteCoef_perm_snd (la lb lb' : List 𝓕)
    (h : lb.Perm lb') :
    superCommuteCoef q la lb = superCommuteCoef q la lb'
```

lemma superCommuteCoef_mul_self [source]

```
lemma superCommuteCoef_mul_self (l lb : List 𝓕) :
    superCommuteCoef q l lb * superCommuteCoef q l lb = 1
```

lemma superCommuteCoef_empty [source]

```
lemma superCommuteCoef_empty (la : List 𝓕) :
    superCommuteCoef q la [] = 1
```

lemma superCommuteCoef_append [source]

```
lemma superCommuteCoef_append (la lb lc : List 𝓕) :
    superCommuteCoef q la (lb ++ lc) = superCommuteCoef q la lb * superCommuteCoef q la lc
```

lemma superCommuteCoef_cons [source]

```
lemma superCommuteCoef_cons (i : 𝓕) (la lb : List 𝓕) :
    superCommuteCoef q la (i :: lb) = superCommuteCoef q la [i] * superCommuteCoef q la lb
```

2.22 HepLean.PerturbationTheory.Wick.StaticTheorem

HepLean/PerturbationTheory/Wick/StaticTheorem.lean on GitHub.

lemma static_wick_nil [source]

```
lemma static_wick_nil {A : Type} [Semiring A] [Algebra ℂ A]
    (F : FreeAlgebra ℂ (Σ i, f i) →ₐ A)
    (S : Contractions.Splitting f le) :
    F (ofListLift f [] 1) = ∑ c : Contractions [],
    c.toCenterTerm f q le F S *
    F (koszulOrder (fun i => q i.fst) le (ofListLift f c.normalize 1))
```

lemma static_wick_cons [source]

```
lemma static_wick_cons [IsTrans ((i : 𝓕) × f i) le] [IsTotal ((i : 𝓕) × f i) le]
    {A : Type} [Semiring A] [Algebra ℂ A] (r : List 𝓕) (a : 𝓕)
    (F : FreeAlgebra ℂ (Σ i, f i) →ₐ A) [OperatorMap (fun i => q i.1) le F]
    (S : Contractions.Splitting f le)
    (ih : F (ofListLift f r 1) =
    ∑ c : Contractions r, c.toCenterTerm f q le F S * F (koszulOrder (fun i => q i.fst) le
      (ofListLift f c.normalize 1))) :
    F (ofListLift f (a :: r) 1) = ∑ c : Contractions (a :: r),
      c.toCenterTerm f q le F S *
      F (koszulOrder (fun i => q i.fst) le (ofListLift f c.normalize 1))
```

theorem static_wick_theorem [source]

```
theorem static_wick_theorem [IsTrans ((i : 𝓕) × f i) le] [IsTotal ((i : 𝓕) × f i) le]
    {A : Type} [Semiring A] [Algebra ℂ A] (r : List 𝓕)
    (F : FreeAlgebra ℂ (Σ i, f i) →ₐ A) [OperatorMap (fun i => q i.1) le F]
    (S : Contractions.Splitting f le) :
    F (ofListLift f r 1) = ∑ c : Contractions r, c.toCenterTerm f q le F S *
    F (koszulOrder (fun i => q i.fst) le (ofListLift f c.normalize 1))
```

2.23 HepLean.PerturbationTheory.Wick.SuperCommute

HepLean/PerturbationTheory/Wick/SuperCommute.lean on GitHub.

def superCommuteMonoidAlgebra [source]

> *Given a grading `q : I → Fin 2` and a list `l : List I` the super-commutor on the free algebra `FreeAlgebra ℂ I` corresponding to commuting with `l` as a linear map from `MonoidAlgebra ℂ (FreeMonoid I)` (the module of lists in `I`) to itself.*

```
def superCommuteMonoidAlgebra (l : List 𝓕) :
    MonoidAlgebra ℂ (FreeMonoid 𝓕) →ₗ[ℂ] MonoidAlgebra ℂ (FreeMonoid 𝓕)
```

def superCommuteAlgebra [source]

Given a grading `q : I → Fin 2` the super-commutor on the free algebra `FreeAlgebra ℂ I` as a linear map from `MonoidAlgebra ℂ (FreeMonoid I)` (the module of lists in `I`) to `FreeAlgebra ℂ I →ₗ[ℂ] FreeAlgebra ℂ I`.

```
def superCommuteAlgebra :
    MonoidAlgebra ℂ (FreeMonoid 𝓕) →ₗ[ℂ] FreeAlgebra ℂ 𝓕 →ₗ[ℂ] FreeAlgebra ℂ 𝓕
```

def superCommute [source]

Given a grading `q : I → Fin 2` the super-commutor on the free algebra `FreeAlgebra ℂ I` as a bi-linear map.

```
def superCommute :
    FreeAlgebra ℂ 𝓕 →ₗ[ℂ] FreeAlgebra ℂ 𝓕 →ₗ[ℂ] FreeAlgebra ℂ 𝓕
```

lemma equivMonoidAlgebraFreeMonoid_freeAlgebra [source]

```
lemma equivMonoidAlgebraFreeMonoid_freeAlgebra {I : Type} (i : I) :
    (FreeAlgebra.equivMonoidAlgebraFreeMonoid (FreeAlgebra.ι ℂ i)) =
    Finsupp.single (FreeMonoid.of i) 1
```

lemma superCommute_ι [source]

```
@[simp]
lemma superCommute_ι (i j : 𝓕) :
    superCommute q (FreeAlgebra.ι ℂ i) (FreeAlgebra.ι ℂ j) =
    FreeAlgebra.ι ℂ i * FreeAlgebra.ι ℂ j +
    if q i = fermionic ∧ q j = fermionic then
      FreeAlgebra.ι ℂ j * FreeAlgebra.ι ℂ i
    else
      - FreeAlgebra.ι ℂ j * FreeAlgebra.ι ℂ i
```

lemma superCommute_ofList_ofList [source]

```
lemma superCommute_ofList_ofList (l r : List 𝓕) (x y : ℂ) :
    superCommute q (ofList l x) (ofList r y) =
    ofList (l ++ r) (x * y) + (if FieldStatistic.ofList q l = fermionic ∧
      FieldStatistic.ofList q r = fermionic then
    ofList (r ++ l) (y * x) else - ofList (r ++ l) (y * x))
```

lemma superCommute_zero [source]

```
@[simp]
lemma superCommute_zero (a : FreeAlgebra ℂ 𝓕) :
    superCommute q a 0 = 0
```

lemma superCommute_one [source]

```
@[simp]
lemma superCommute_one (a : FreeAlgebra ℂ 𝓕) :
    superCommute q a 1 = 0
```

lemma superCommute_ofList_mul [source]

```
lemma superCommute_ofList_mul (la lb lc : List 𝓕) (xa xb xc : ℂ) :
    superCommute q (ofList la xa) (ofList lb xb * ofList lc xc) =
    (superCommute q (ofList la xa) (ofList lb xb) * ofList lc xc +
    superCommuteCoef q la lb • ofList lb xb * superCommute q (ofList la xa) (ofList lc xc))
```

def superCommuteSplit [source]

Given two lists `la lb : List I`, in the expansion of the supercommutor of `la` and `lb` via elements of `lb`the term associated with the `n`th element. E.g. in the commutator `[a, bc] = [a, b] c + b [a, c]` the `superCommuteSplit` for `n=0` is `[a, b] c` and for `n=1` is `b [a, c]`.

```
def superCommuteSplit (la lb : List 𝓕) (xa xb : ℂ) (n : ℕ)
    (hn : n < lb.length) : FreeAlgebra ℂ 𝓕
```

lemma superCommute_ofList_cons [source]

```
lemma superCommute_ofList_cons (la lb : List 𝓕) (xa xb : ℂ) (b1 : 𝓕) :
    superCommute q (ofList la xa) (ofList (b1 :: lb) xb) =
    superCommute q (ofList la xa) (FreeAlgebra.ι ℂ b1) * ofList lb xb +
    superCommuteCoef q la [b1] •
    (ofList [b1] 1) * superCommute q (ofList la xa) (ofList lb xb)
```

lemma superCommute_ofList_sum [source]

```
lemma superCommute_ofList_sum (la lb : List 𝓕) (xa xb : ℂ) :
    superCommute q (ofList la xa) (ofList lb xb) =
    ∑ (n : Fin lb.length), superCommuteSplit q la lb xa xb n n.prop
```

lemma superCommute_ofList_ofList_superCommuteCoef [source]

```
lemma superCommute_ofList_ofList_superCommuteCoef (la lb : List 𝓕)
    (xa xb : ℂ) : superCommute q (ofList la xa) (ofList lb xb) =
    ofList la xa * ofList lb xb - superCommuteCoef q la lb • ofList lb xb * ofList la xa
```

lemma ofList_ofList_superCommute [source]

```
lemma ofList_ofList_superCommute (la lb : List 𝓕) (xa xb : ℂ) :
    ofList la xa * ofList lb xb = superCommuteCoef q la lb • ofList lb xb * ofList la xa
    + superCommute q (ofList la xa) (ofList lb xb)
```

lemma ofListLift_ofList_superCommute' [source]

```
lemma ofListLift_ofList_superCommute' (l : List 𝓕) (r : List 𝓕) (x y : ℂ) :
    ofList r y * ofList l x = superCommuteCoef q l r • (ofList l x * ofList r y)
    - superCommuteCoef q l r • superCommute q (ofList l x) (ofList r y)
```

lemma superCommute_ofList_ofListLift [source]

```
lemma superCommute_ofList_ofListLift (l : List (Σ i, f i)) (r : List 𝓕) (x y : ℂ) :
    superCommute (fun i => q i.1) (ofList l x) (ofListLift f r y) =
```

```
    ofList l x * ofListLift f r y +
    (if FieldStatistic.ofList (fun i => q i.1) l = fermionic ∧
      FieldStatistic.ofList q r = fermionic then
    ofListLift f r y * ofList l x else - ofListLift f r y * ofList l x)
```

lemma superCommute_ofList_ofListLift_superCommuteLiftCoef [source]

```
lemma superCommute_ofList_ofListLift_superCommuteLiftCoef
    (l : List (Σ i, f i)) (r : List 𝓕) (x y : ℂ) :
    superCommute (fun i => q i.1) (ofList l x) (ofListLift f r y) =
    ofList l x * ofListLift f r y - superCommuteLiftCoef q l r • ofListLift f r y * ofList l x
```

lemma ofList_ofListLift_superCommute [source]

```
lemma ofList_ofListLift_superCommute (l : List (Σ i, f i)) (r : List 𝓕) (x y : ℂ) :
    ofList l x * ofListLift f r y = superCommuteLiftCoef q l r • ofListLift f r y * ofList l x
    + superCommute (fun i => q i.1) (ofList l x) (ofListLift f r y)
```

lemma ofListLift_ofList_superCommute [source]

```
lemma ofListLift_ofList_superCommute (l : List (Σ i, f i)) (r : List 𝓕) (x y : ℂ) :
    ofListLift f r y * ofList l x = superCommuteLiftCoef q l r • (ofList l x * ofListLift f r y
    )
    - superCommuteLiftCoef q l r •
    superCommute (fun i => q i.1) (ofList l x) (ofListLift f r y)
```

lemma superCommuteLiftCoef_append [source]

```
lemma superCommuteLiftCoef_append (l : List (Σ i, f i)) (r1 r2 : List 𝓕) :
    superCommuteLiftCoef q l (r1 ++ r2) =
    superCommuteLiftCoef q l r1 * superCommuteLiftCoef q l r2
```

def superCommuteLiftSplit [source]

Given two lists `l : List (Σ i, f i)` and `r : List I`, on in the expansion of the supercommutor of `l` and the lift of `r` via elements of `r`the term associated with the `n`th element. E.g. in the commutator `[a, bc] = [a, b] c + b [a, c]` the `superCommuteSplit` for `n=0` is `[a, b] c` and for `n=1` is `b [a, c]`.

```
def superCommuteLiftSplit (l : List (Σ i, f i)) (r : List 𝓕) (x y : ℂ) (n : ℕ)
    (hn : n < r.length) : FreeAlgebra ℂ (Σ i, f i)
```

lemma superCommute_ofList_ofListLift_cons [source]

```
lemma superCommute_ofList_ofListLift_cons (l : List (Σ i, f i)) (r : List 𝓕) (x y : ℂ) (b1 :
    𝓕) :
    superCommute (fun i => q i.1) (ofList l x) (ofListLift f (b1 :: r) y) =
    superCommute (fun i => q i.1) (ofList l x) (sumFiber f (FreeAlgebra.ι ℂ b1))
    * ofListLift f r y + superCommuteLiftCoef q l [b1] •
    (ofListLift f [b1] 1) * superCommute (fun i => q i.1) (ofList l x) (ofListLift f r y)
```

lemma superCommute_ofList_ofListLift_sum [source]

```
lemma superCommute_ofList_ofListLift_sum (l : List (Σ i, f i)) (r : List 𝓕) (x y : ℂ) :
    superCommute (fun i => q i.1) (ofList l x) (ofListLift f r y) =
    ∑ (n : Fin r.length), superCommuteLiftSplit q l r x y n n.prop
```

2.24 HepLean.Tensors.OverColor.Functors

HepLean/Tensors/OverColor/Functors.lean on GitHub.

instance map_laxMonoidal [source]

The functor `map f` is lax-monoidal.

```
instance map_laxMonoidal {C D : Type} (f : C → D) : Functor.LaxMonoidal (map f) where
```

instance map_monoidal [source]

The functor `map f` is lax-monoidal.

```
noncomputable instance map_monoidal {C D : Type} (f : C → D) : Functor.Monoidal (map f)
```

instance tensor_laxMonoidal [source]

The functor tensor is lax-monoidal.

```
instance tensor_laxMonoidal (C : Type) : Functor.LaxMonoidal (tensor C) where
```

instance tensor_monoidal [source]

The functor tensor is monoidal.

```
noncomputable instance tensor_monoidal (C : Type) : Functor.Monoidal (tensor C)
```

instance const_laxMonoidal [source]

The functor `const C` is lax monoidal.

```
instance const_laxMonoidal (C : Type) : Functor.LaxMonoidal (const C) where
```

instance const_monoidal [source]

The functor `const C` is monoidal.

```
noncomputable instance const_monoidal (C : Type) : Functor.Monoidal (const C)
```

instance contrPair_laxMonoidal [source]

The functor `contrPair` is lax-monoidal.

```
instance contrPair_laxMonoidal (C : Type) (τ : C → C) : Functor.LaxMonoidal (contrPair C τ)
```

instance contrPair_monoidal [source]

The functor `contrPair` is monoidal.

```
noncomputable instance contrPair_monoidal (C : Type) (τ : C → C) :
    Functor.Monoidal (contrPair C τ)
```

2.25 HepLean.Tensors.OverColor.Lift

HepLean/Tensors/OverColor/Lift.lean on GitHub.

instance obj'_laxBraidedFunctor [source]

The lift of a functor is lax braided.

```
instance obj'_laxBraidedFunctor : Functor.LaxBraided (obj' F) where
```

instance obj'_monoidalFunctor [source]

The lift of a functor is monoidal.

```
instance obj'_monoidalFunctor : Functor.Monoidal (obj' F)
```

instance obj'_braided [source]

The lift of a functor is braided.

```
instance obj'_braided : Functor.Braided (obj' F)
```

instance map'_isMonoidal [source]

The lift of a natural transformation is monoidal.

```
instance map'_isMonoidal : NatTrans.IsMonoidal (map' η) where
```

instance (lift.obj F).Monoidal [source]

The lift of a functor is monoidal.

```
noncomputable instance : (lift.obj F).Monoidal
```

instance (lift.obj F).LaxBraided [source]

The lift of a functor is lax-braided.

```
noncomputable instance : (lift.obj F).LaxBraided
```

instance (lift.obj F).Braided [source]

The lift of a functor is braided.

```
noncomputable instance : (lift.obj F).Braided
```

2.26 HepLean.Tensors.TensorSpecies.Basic

HepLean/Tensors/TensorSpecies/Basic.lean on GitHub.

instance F_monoidal [source]

The functor `F` is monoidal.

```
instance F_monoidal : Functor.Monoidal S.F
```

instance F_laxBraided [source]

The functor `F` is lax-braided.

```
instance F_laxBraided : Functor.LaxBraided S.F
```

instance F_braided [source]

The functor `F` is braided.

```
instance F_braided : Functor.Braided S.F
```

2.27 scripts.MetaPrograms.notes

scripts/MetaPrograms/notes.lean on GitHub.

def pertubationTheory [source]

```
def pertubationTheory : NoteFile where
```

2.28 scripts.MetaPrograms.redundent_imports

scripts/MetaPrograms/redundent_imports.lean on GitHub.

def RedundentImports [source]

```
def Imports.RedundentImports (imp : Import) : MetaM UInt32
```